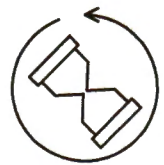


CHAPTER

15

General-purpose Timers

Timers are special hardware components that provide accurate timestamps, time-interval measurements, and timer-related periodic events for both hardware and software. This chapter presents two example uses of timers: measuring the pulse lengths of input signals (input capture) or generating output waveforms (output compare and PWM).



15.1 Timer Organization and Counting Modes

A timer is a free-run hardware counter that increments or decrements once for every clock cycle. The counter runs continuously until the timer is disabled. The counting process restarts automatically when the counter reaches 0 during down-counting or some maximum value during up-counting. Software can select the frequency of the timer clock so that the free-run counter increments or decrements at some desired speed.

If a timer works as *output compare*, as shown in Figure 15-1, the comparator consistently compares the counter value with some given constant, and generates an output or an interrupt if they are equal. Software can program the constant value to control the timing of outputs or interrupts.

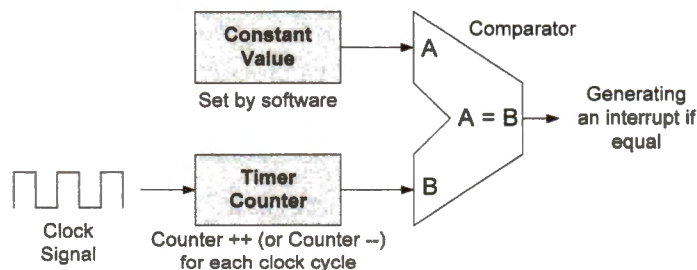


Figure 15-1. A timer is used as output compare. The timer counter is a hardware register, and it automatically increments or decrements by 1 for each clock cycle.

If the function of a timer is *input capture* (see Figure 15-2), the hardware automatically logs the counter value into a special register (called CCR) and generates an interrupt when the desired event occurs. Typically, the interrupt handler needs to copy register CCR to a user buffer to record the timing of past events. Then, software calculates the difference between two logged values and finds the time span of two events.

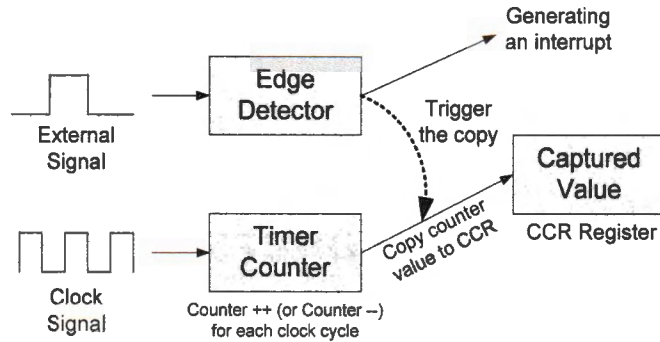


Figure 15-2. A timer is used as input capture. The edge detector triggers the hardware to copy timer counter to register CCR on rising or falling edges of monitored signal.

The hardware timer counter has three different counting modes: *up-counting*, *down-counting*, or *center-aligned counting*, as illustrated in Figure 15-3.

- In the *up-counting* mode, the counter starts from 0 to a constant and then restarts from 0. Software sets up the constant and stores it in a special register called the auto-reload register (ARR). For example, if ARR is 4, the counter value is 0, 1, 2, 3, 4, 0, 1, 2, 3, 4, and repeats until the timer is disabled.
- In the *down-counting* mode, the counter starts from the auto-reload value down to 0 and then restarts from the auto-reload value. For example, if ARR is 4, the counter value is 4, 3, 2, 1, 0, 4, 3, 2, 1, 0, and repeats until the timer is disabled.
- The third one is the *center-aligned counting* mode, which performs up-counting and down-counting alternatively. For example, if ARR is 4, the counter value is 0, 1, 2, 3, 4, 3, 2, 1, 0, and repeats until the timer is disabled.

The timer counter forms a periodical sawtooth or triangle wave. The period is controlled by both the clock frequency to the counter ($f_{\text{CLOCK_CNT}}$) and the value stored in the ARR register. For up-counting and down-counting, the counting period of the sawtooth waveform is

$$\text{Counting Period of Sawtooth Waveform} = (1 + \text{ARR}) \times \frac{1}{f_{\text{CLOCK_CNT}}}$$

For center-aligned counting, the counting period of the triangle waveform is

$$\text{Counting Period of Triangle Waveform} = 2 \times \text{ARR} \times \frac{1}{f_{\text{CLOCK_CNT}}}$$

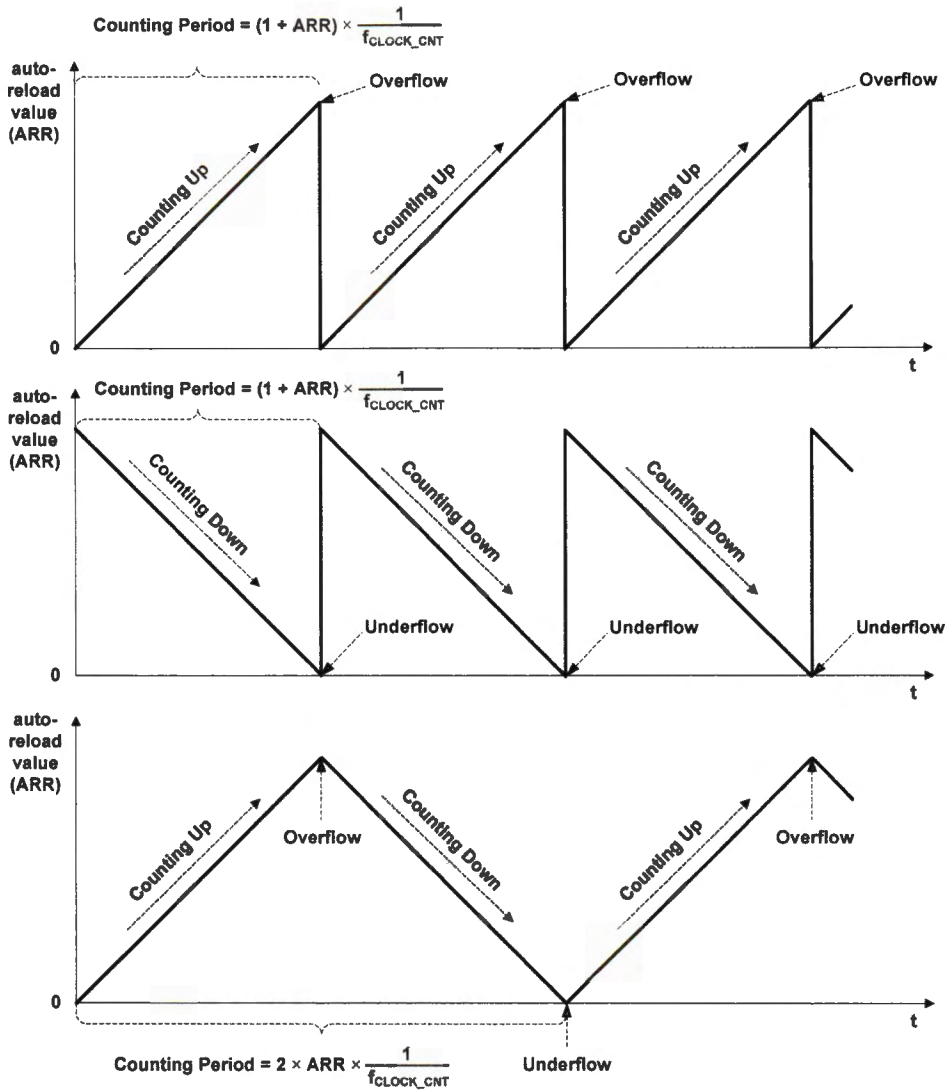


Figure 15-3. Three counting modes: up-counting, down-counting, and center-aligned counting

A timer counter has two update events: *overflow* and *underflow*, as shown in Figure 15-3. In the up-counting mode, overflow occurs when the counter is reset to 0. In the down-counting mode, underflow occurs when the counter is reset to ARR. In the center-aligned counting mode, underflow and overflow occur alternatively.

When using the timer to measure a large time span between the occurrences of two events, software must consider the overflow and underflow to avoid underestimating the time span. The timer interrupt handler can check appropriate flags in the timer's status register to count how many times overflow or underflow has occurred.

15.2 Compare Output

Figure 15-4 shows the basic diagram of the output compare of a timer. The timer counter (CNT) has 16 bits. The capture/compare register (CCR) holds the value that is compared with the timer counter.

In STM32L, four output channels share the same free-run timer counter. Therefore, the timer hardware compares the timer counter with four CCR registers simultaneously and generates four independent outputs based on the comparison results.

The clock to drive the timer counter (*CLOCK_CNT*) can be slowed down by a constant factor called *prescaler* to generate output that spans over a long period.

$$f_{CLOCK_CNT} = \frac{f_{CLOCK_PSC}}{Prescaler + 1}$$

A large prescaler reduces the timer's resolution, but decreases the chance of overflow and underflow and improves the energy efficiency.

Different clocks can drive the timer. These clocks include built-in clocks within the processor chip, external crystal oscillators, or some internal trigger signal such as the output of another timer. External clocks are preferred over internal clocks because external clocks are more accurate than internal clocks.

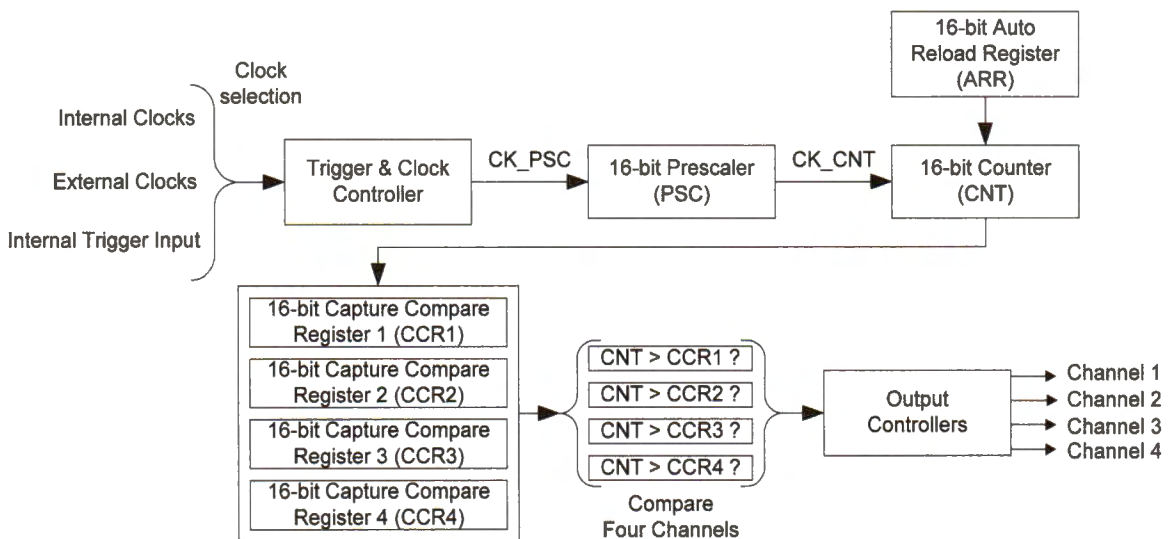


Figure 15-4. Output compare diagram of a timer with four channels. Some timers have more than four channels, with extra channels used internally, such as for triggering ADC.

15.2.1 Setting Output Mode

When the timer counter (CNT) equals the compare value (CCR), the output of a channel (OCREF) is programmable. The output can have different values, depending on the output compare mode (OCM), as shown in Table 15-1.

Output Compare Mode (OCM)	Timer Reference Output (OCREF)
Timing mode (0000)	Frozen
Active mode (0001)	Logic high if CNT = CCR
Inactive mode (0010)	Logic low if CNT = CCR
Toggle mode (0011)	Toggle if CNT = CCR
Forced inactive mode (0100)	Forced logic low (always low)
Forced active mode (0101)	Forced logic high (always high)
PWM output mode 1 (0110)	In up-counting: Logic high if CNT < CCR, else logic low In down-counting: Logic high if CNT ≤ CCR, else logic low
PWM output mode 2 (0111)	In up-counting: Logic high if CNT ≥ CCR, else logic low In down-counting: Logic high if CNT > CCR, else logic low

Table 15-1. Control of timer channel output

The active and inactive mode outputs a logic high and logic low, respectively, when the free-run counter CNT matches the capture and compare register (CCR). The toggle mode reverses the output whenever CNT and CCR match, making the output switch between logic high and logic low alternatively. The forced inactive and active mode makes the output stay low and high, respectively.

In digital circuits, there are two options to represent logic values: *active high* and *active low*, as shown in the table below. Software can change the logic representation for the output of each timer channel independently by programming the output polarity bit in the control register CCER. When the polarity bit is 0, active high is chosen to generate voltage outputs. Otherwise, active low is used.

	Active High Signals	Active Low Signals
Logic High (1)	High voltage	Low voltage
Logic Low (0)	Low voltage	High voltage

Figure 15-5 shows the channel reference output OCREF when the output mode is toggled, non-one-pulse high, and one-pulse high. In this example, the counter repeatedly counts up from 0 to ARR. Active high is always used for OCREF. However, depending on the polarity selection, the channel output is either OCREF or the negation of OCREF.

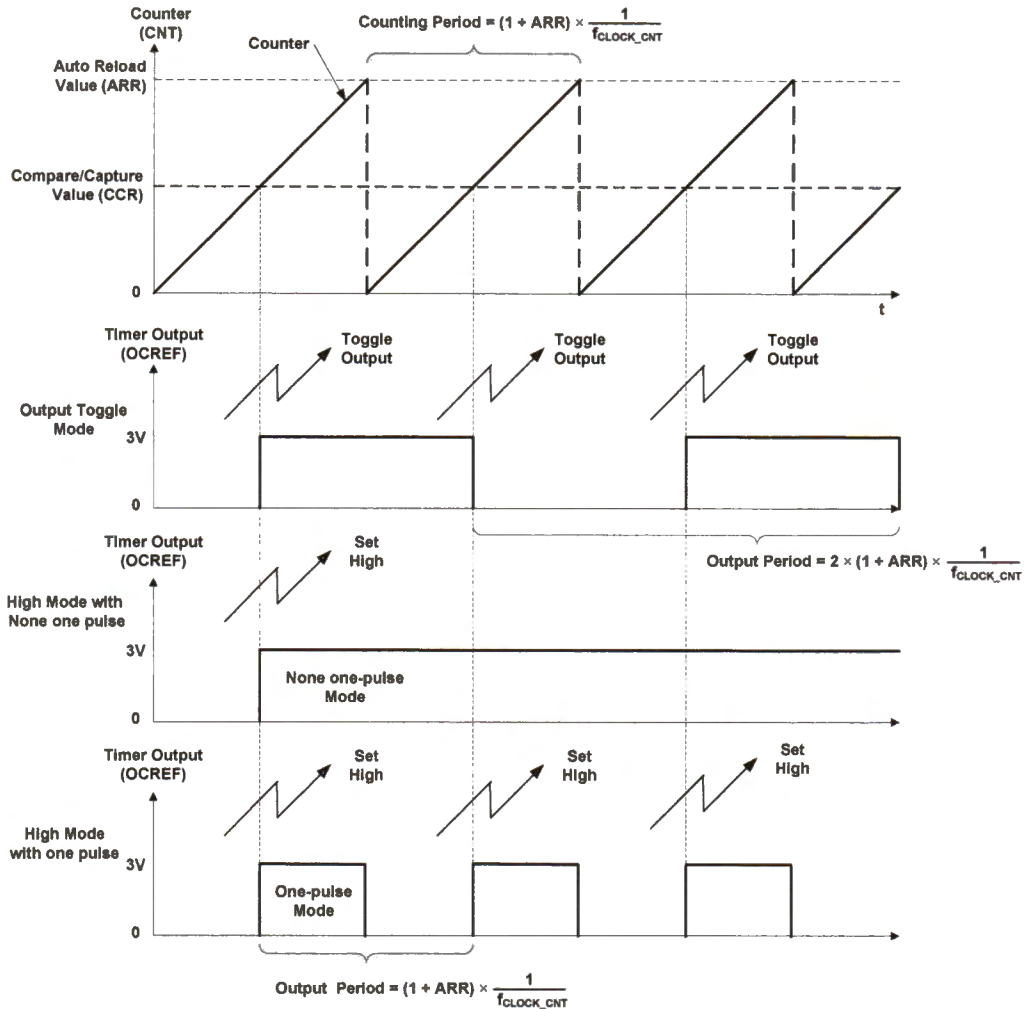


Figure 15-5. OCREF output in the mode of toggle, non-one-pulse high, and one-pulse high.

A timer channel may have two outputs: main output OC and complementary output OCN, which are the exclusive-OR between the channel reference output OCREF and its corresponding polarity bit in the CCER register, as shown below. The CCP and CCNP bits in the CCER register are the polarity bit of OC and OCN, respectively.

- If only OC or OCN is enabled:

$$\begin{aligned} OC &= OCREF + \text{Polarity bit for OC} \\ OCN &= OCREF + \text{Polarity bit for OCN} \end{aligned}$$

- If both OC and OCN are enabled:

$$\begin{aligned} OC &= OCREF + \text{Polarity bit for OC} \\ OCN &= (\text{not } OCREF) + \text{Polarity bit for OCN} \end{aligned}$$

Active high logic is always used for OCREF. However, OC and OCN can be either active high or active low, depending on their polarity bits. If the polarity bit is 0, the corresponding channel output is active high. Otherwise, it is active low.

If timer interrupt is enabled, an interrupt is generated if CNT matches CCR, or CNT has an overflow or underflow. The timer interrupt service routine must check the timer status register to find what event has occurred. The update interrupt flag (UIF) is set for an overflow or underflow, and the capture and compare interrupt flag (CCIF) is set when CNT matches CCR.

A separate DMA interrupt can also be generated to load a value stored in the data memory into ARR or CRR automatically. Chapter 19 introduces DMA operations.

15.2.2 Example of Toggling LED

This section uses the output compare mode of a timer to toggle a green LED attached GPIO pin PE6 every second, as shown below.

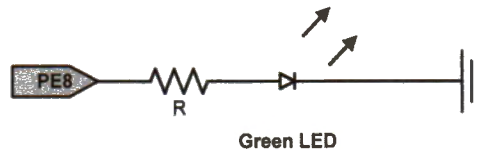


Figure 15-6. The green LED is connected to pin PE 8 on STM32L4 Discovery Kit.

Each GPIO pin can perform multiple hardware functions. Functions that a pin supports vary among chips and manufacturers, but also differ between pins. Due to chip complexity and costs, it is impractical for a pin to support all functions. For STM32L4, the functions that pin PE8 supports are shown in Table 15-2. In this example, we use the alternative function 1 (timer 1, complementary output of channel 1) to toggle the LED.

PIN	AF1	AF6	AF12	AF13	AF15
PE8	TIM1_CH1N	DFSDM_CKIN2	FMC_D5	SAI1_SCK_B	EVENTOUT

Table 15-2. Available alternate functions for PE 8 on STM32L4

Suppose the system clock is 80 MHz and it is selected as the timer clock that drives the timer 1. driving the timer is 2.097 MHz. The following calculates the prescaler that slows down the clock of the timer counter to 2 KHz. Since we have

$$f_{\text{CLOCK_CNT}} = \frac{f_{\text{CLOCK_PSC}}}{\text{Prescaler} + 1}$$

We get

$$\text{Prescaler} = \frac{f_{\text{CLOCK_PSC}}}{f_{\text{CLOCK_CNT}}} - 1 = \frac{80\text{MHz}}{2\text{KHz}} - 1 = 40000 - 1 = 39999$$

To turn an LED on for 1 second and then off for 1 second repeatedly, the auto-reload register (ARR) should be 1999 because the clock frequency of the free-run counter is 2 KHz. The timer counts from 0 to 1999 repeatedly, taking 2000 cycles in each counting period. Register CCR can be set to any integer value between 0 and 1999.

The following is the C implementation. Software sets TIM1_CH1N (the complementary output of channel 1 of timer 1) as compare output and toggles the external LED connected to pin PE 8 every second.

```
int main() {

    System_Clock_Init(); // Switch System Clock = 80 MHz

    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOEEN; // Enable GPIOE clock

    // Set mode of pin 8 as alternate function
    // 00 = Input, 01 = Output, 10 = Alternate Function, 11 = Analog
    GPIOE->MODER &= ~(3UL << 16); // Clear bit 17 and bit 16
    GPIOE->MODER |= 2UL << 16; // Set mode as 10

    // Select alternate function 1 (TIM1_CH1N). See Appendix I
    GPIOE->AFR[1] &= ~(0xF); // ARF[0] for pin 0-7, ARF[1]: pin 8-15
    GPIOE->AFR[1] |= 1UL; // TIM1_CH1N defined as 01

    // Set I/O output speed value as Low
    // 00 = Low, 01 = Medium, 10 = Fast, 11 = High
    GPIOE->OSPEEDR &= ~(3UL << 16);

    // Set pin PE8 as no pull-up/pull-down
    // 00 = No PUPD, 01 = Pull up, 10 = Pull down, 11 = Reserved
    GPIOE->PUPDR &= ~(3UL << 16);

    // Enable timer 1 clock
    RCC->APB2ENR |= RCC_APB2ENR_TIM1EN;

    // Counting direction: 0 = up-counting, 1 = down-counting
    TIM1->CR1 &= ~TIM_CR1_DIR;

    // Clock prescaler (16 bits, up to 65,535)
    TIM1->PSC = 39999;

    // Auto-reload: up-counting (0-->ARR), down-counting (ARR-->0)
    TIM1->ARR = 2000-1;

    // Can be any value between 0 and 1999.
    TIM1->CCR1 = 500;

    // Main output enable (MOE): 0 = Disable, 1 = Enable
    TIM1->BDTR |= TIM_BDTR_MOE;

    // Clear output compare mode bits for channel 1
    TIM1->CCMR1 &= ~TIM_CCMR1_OC1M;
```



```

// Select toggle mode (0011)
TIM1->CCMR1 |= TIM_CCMR1_OC1M_0 | TIM_CCMR1_OC1M_1;

// Select output polarity: 0 = active high, 1 = active low
TIM1->CCER &= ~TIM_CCER_CC1NP; // select active high

// Enable output for channel 1 complementary output
TIM1->CCER |= TIM_CCER_CC1NE;

// Enable timer 1
TIM1->CR1 |= TIM_CR1_CEN;

while(1); // dead Loop
}

```

Example 15-1. Lighting up an LED by using the compare-output function of a timer

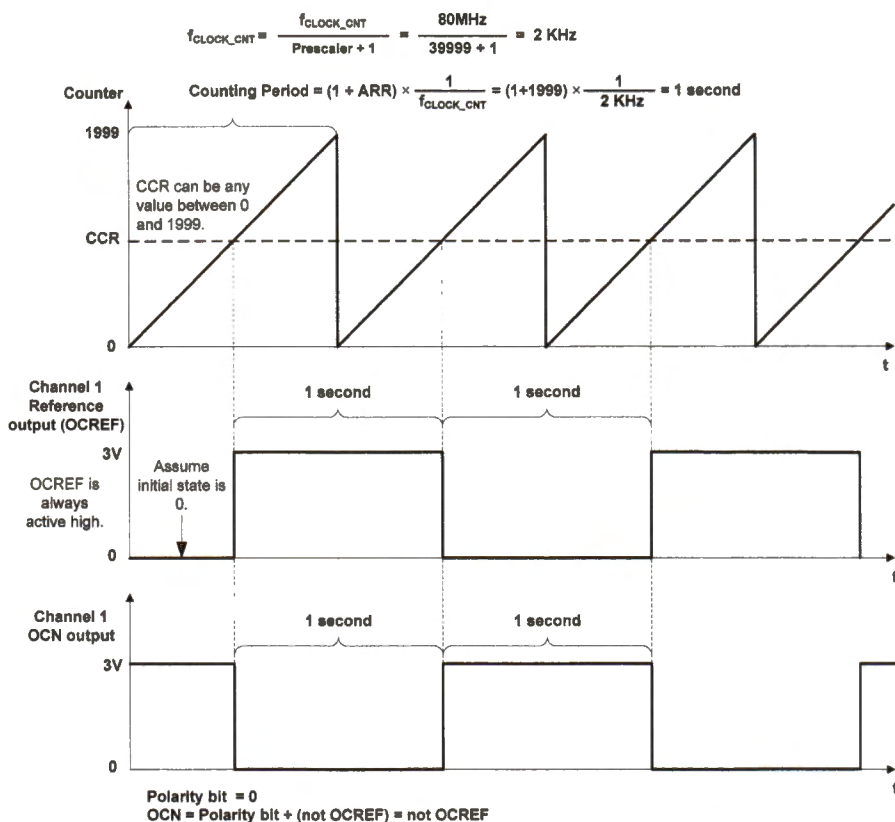


Figure 15-7. Voltage output signal on pin PE 8

Figure 15-7 shows the signals of the timer counter, the reference output (OCREF), and the complementary output (OCN) of channel 1. The initial state of OCREF is assumed to be 0 or low. If the initial state is critical, software can use the forced output mode to force OCREF to 1 (high) or 0 (low). Note that OCREF always uses active high logic.

15.2.3 Timer Update Events

An update event (UEV) is generated on each overflow in up-counting, on each underflow in down-counting, on both overflow and underflow in center-counting.

$$UEV\ Period = (1 + ARR) \times (1 + Prescaler) \times \frac{1}{f_{CK_CNT}}$$

UEV events serve three purposes:

- Generate trigger output (TRGO) for other internal modules, such as timers, DMA, ADC, and DAC.
- Make updates of registers ARR, PSC, and CCR take effect immediately if the buffering (also called preload) mechanism is enabled. If the channel's preload enable bit (OCPE) in the CCMR1 register and the auto-reload preload enable bit (ARPE) in the CR1 register are set, the preload mechanism is enabled.
- Generate a timer interrupt request if the update interrupt flag bit (UIF) of the control register CR1 is set. Interrupt requests are sent to the interrupt controller (NVIC). In response, the processor executes the corresponding interrupt handler.

Software can disable UEV by setting the update disable bit (UDIS) in the CR1 register. If so, update events are not generated.

Software can also generate UEV events. If the update request selection bit (URS) in the CR1 register is set, setting the update generation bit (UG) in the event generation register (EGR) generates an UEV event.

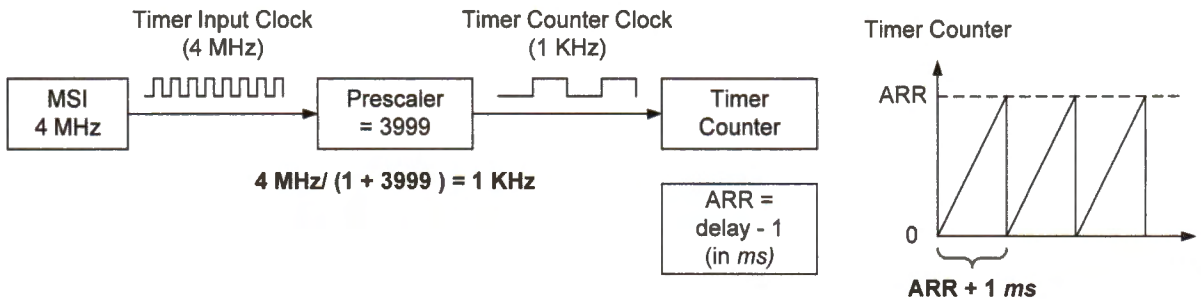


Figure 15-8. Implementation of hardware-precision delay

As shown in Example 15-2, software can use the timer update events to achieve hardware-precision delay. At 30 °C, MSI and HSI can reach an accuracy of $\pm 0.6\%$. Assume MSI 4 MHz is selected to drive the timer, the prescaler is set to 3999, and the timer counters are in the up-counting mode, as shown in Figure 15-8. Thus, the timer counter is incremented by 1 every millisecond. Timer 7 is selected because it is a basic timer without advanced functions. We reserve advanced timers for complicate usage.

```

void delay(uint16_t ms) {

    if ( ms == 0 ) // Sanity check
        return;

    // Enable timer 7 clock
    RCC->APB1ENR1 |= RCC_APB1ENR1_TIM7EN;

    TIM7->CR1 &= ~TIM_CR1_CEN; // Disable timer 7
    TIM7->SR = 0;                // Clear status register
    TIM7->CNT = 0;                // Clear counter
    TIM7->PSC = 3999;             // 4 MHz/(1 + 3999) = 1 KHz
    TIM7->ARR = ms - 1;           // ARR + 1 cycles
    TIM7->CR1 |= TIM_CR1_CEN;    // Enable timer 7

    // Loop until UIF is set
    while ( (TIM7->SR & TIM_SR_UIF) == 0 );
}

```

Example 15-2. Use timer updates to implement hardware-precision delay

Example 15-3 counts the number of overflows that have taken place on up-counting. If the UIF flag in the timer's status register (SR) is set, the timer interrupt handler increments the overflow counter and clears the flag bit UIF to 0 to prevent the interrupt handler from executing again.

```

volatile uint32_t overflow = 0;

int main(void) {

    // Initialization
    ...

    // Select counting direction
    // 0 = up-counting, 1 = down-counting
    TIM1->CR1 &= ~TIM_CR1_DIR;

    // Enable update interrupts
    TIM1->DIER |= TIM_DIER_UIE;

    // Enable TIM4 interrupt in NVIC
    NVIC_EnableIRQ(TIM1_IRQn);

    // Enable the counter
    TIM1->CR1 |= TIM_CR1_CEN;

    while(1);
}

```

```
void TIM1_IRQHandler(void) {  
  
    // Check whether an overflow event has taken place  
    if((TIM1->SR & TIM_SR_UIF) != 0) {  
  
        // Increment the overflow counter  
        overflow++;  
  
        // Clear flag UIF to prevent reentrance  
        TIM1->SR &= ~TIM_SR_UIF;  
    }  
  
    ...  
}
```

Example 15-3. Counting the number of update events

In this example, the global variable `overflow` is declared as “volatile”. This is to inform the compiler that no optimization should be made on this variable. While the compiler can observe that the interrupt handler changes the value of this variable, it is still possible that the compiler incorrectly reuses an old value if `overflow` was not volatile. The reason is that the compiler notices that the interrupt handler does not appear to be called in software and mistakenly assumes that the value of `overflow` would not change.

volatile: characterized by
or subject to rapid or
unexpected change

-- Merriam-Webster
Dictionary

15.3 PWM Output

Pulse width modulation (PWM) is a simple digital technique to control the value of an analog variable. PWM uses a rectangular waveform to quickly switch a voltage source on and off to produce a desired average voltage output. Although the output is binary at any time instant, the average output over a time span can be any value between 0 and the maximum voltage.

Specifically, the percentage of time in the on state within one period is proportional to the mean value of the voltage output. Consequently, when software changes the duration of the on state, the output voltage is adjusted accordingly to emulate an analog signal.

PWM has been widely used in a variety of applications, such as motor speed and torque control, digital encoding in telecommunications, DC-to-DC power conversion, and audio amplification. This section uses PWM to control the brightness of an LED.



We should select the PWM switching frequency carefully to avoid serious negative impacts on applications. For example, the PWM switching frequency of an LED light must be at least 120 Hz to prevent the flickering effects that humans can see.

The average value of a simple PWM output based on a sawtooth carrier signal and a constant reference, as illustrated in Figure 15-9 and Figure 15-10, is linearly proportional to the *duty cycle*.

The duty cycle is defined as follows:

$$\begin{aligned} \text{duty cycle} &= \frac{\text{pulse on time } (T_{on})}{\text{pulse switching period } (T_s)} \times 100\% \\ &= \frac{T_{on}}{T_{on} + T_{off}} \times 100\% \end{aligned}$$

where

$$\text{pulse switching period} = \frac{1}{\text{PWM switching frequency}}$$

By changing the duty cycle, software can control this average value. In the LED example, the brightness is determined by the PWM duty cycle. Figure 15-9 and Figure 15-10 give three examples in which the average output is 1/6 and 1/2, respectively.

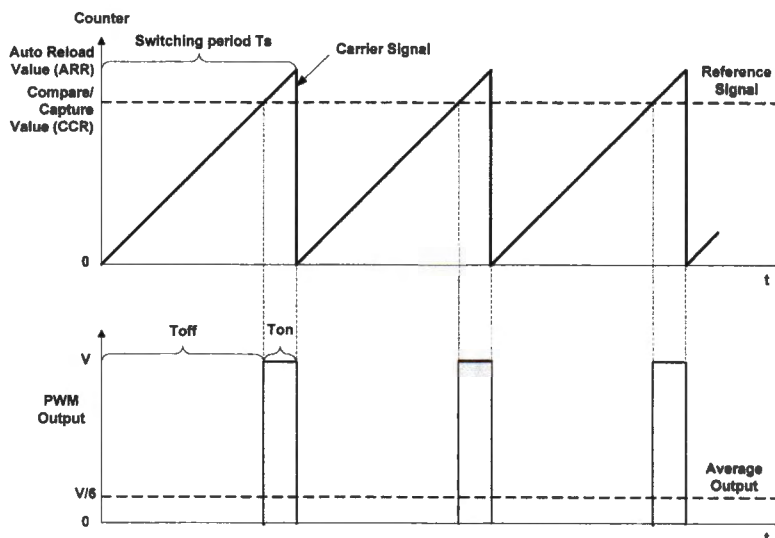


Figure 15-9. Example of simple PWM when duty cycle is 1/6

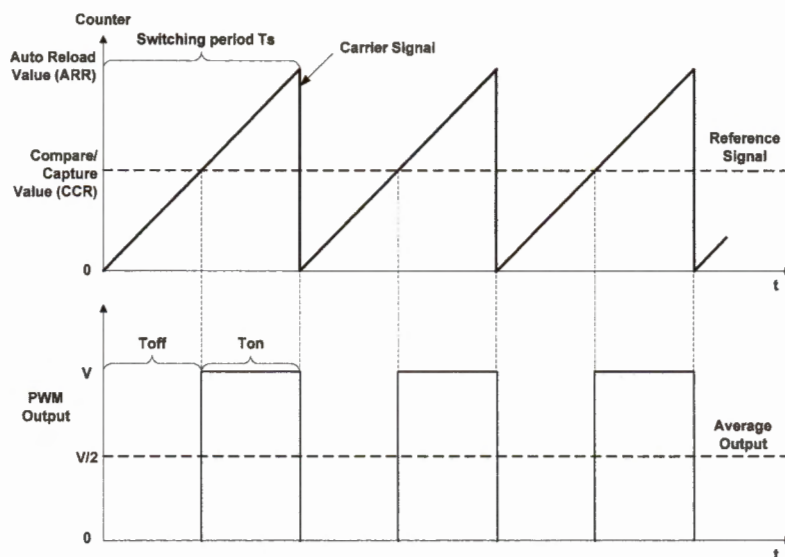


Figure 15-10. Example of simple PWM when duty cycle is 1/2

The PWM output signal is determined by three factors:

1. comparison between the timer counter (CNT) and the given reference valued stored in the compare and capture register (CCR),
2. the PWM output mode, and
3. the polarity bit.

As shown in Table 15-3, there are two PWM modes, which are opposite to each other.

- **PWM Mode 1:** If the counter is less than the reference signal, the timer reference output (OCREF) is then held at logic high; otherwise, it is held at logic low.
- **PWM Mode 2:** The timer reference output (OCREF) in mode 2 is the opposite of mode 1. If the counter is greater than the reference signal, OCREF is then held at active; otherwise, OCREF is held at inactive.

PWM Mode	Counting Mode	PWM Reference Output (OCREF)	
		Logic High	Logic Low
Mode 1	Up-counting	$CNT < CCR$	$CNT \geq CCR$
	Down-counting	$CNT \leq CCR$	$CNT > CCR$
Mode 2	Up-counting	$CNT \geq CCR$	$CNT < CCR$
	Down-counting	$CNT > CCR$	$CNT \leq CCR$

Table 15-3. Timer reference output (OCREF) of PWM mode 1 and mode 2

OCREF is an internal output, which always uses active high logic. However, the actual output (OC or OCN) can be *active high* or *active low*.

- In *active high*, a high voltage represents logic high (or called active), and a low voltage represents logic low (or called inactive).
- The output of *active low* is the opposite of active high.
- Selection of active high or active low is controlled by the polarity bit.

In sum, the actual PWM output (OC or OCN) is determined by both the PWM mode and the polarity bit.

Figure 15-11 shows an example of the timer reference output (OCREF) for center-counting.

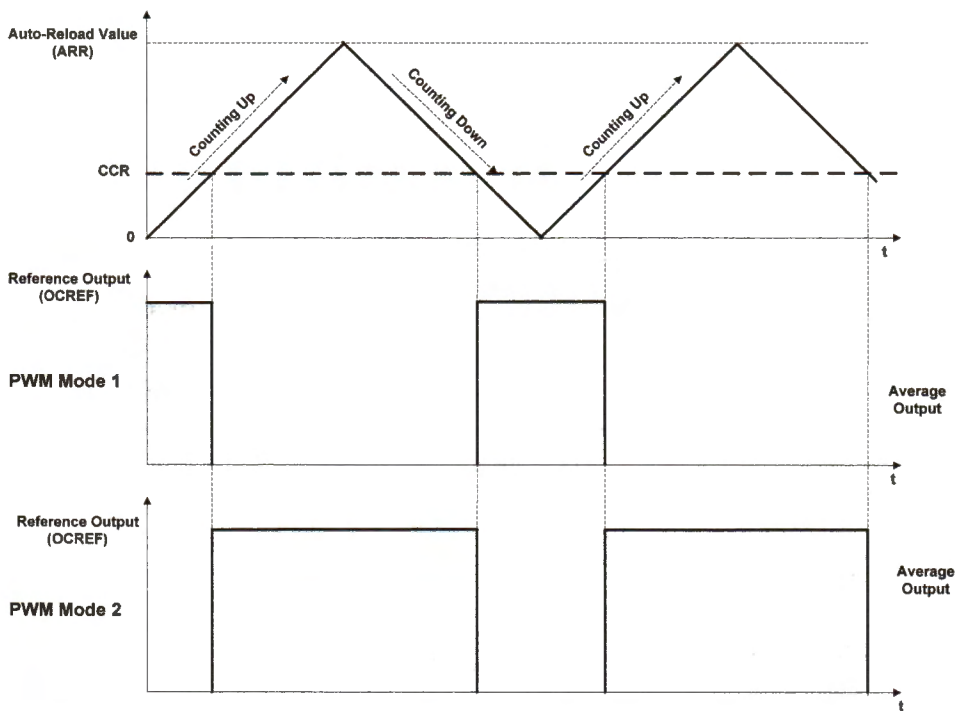


Figure 15-11. Reference output (OCREF) in PWM mode 1 and PWM mode 2

PWM Signal Period:

For up- or down-counting:

$$PWM\ Period = (1 + ARR) \times \frac{Clock\ Period\ of\ Timer}{1 + Prescaler}$$

For center-counting mode:

$$PWM\ Period = 2 \times ARR \times \frac{Clock\ Period\ of\ Timer}{1 + Prescaler}$$

PWM Duty Cycle:

Depending on the PWM mode and the polarity bit, the duty cycle of the main output OC or the complementary output OCN in up-counting or down-counting is

$$\text{Duty Cycle} = \begin{cases} \frac{CCR}{ARR + 1} \\ 1 - \frac{CCR}{ARR + 1} \end{cases}$$

Figure 15-12 gives an example of PWM mode 1 output on up-counting. The duty cycle of the main output (OC) is either 3/7 or 4/7, depending on polarity bit.

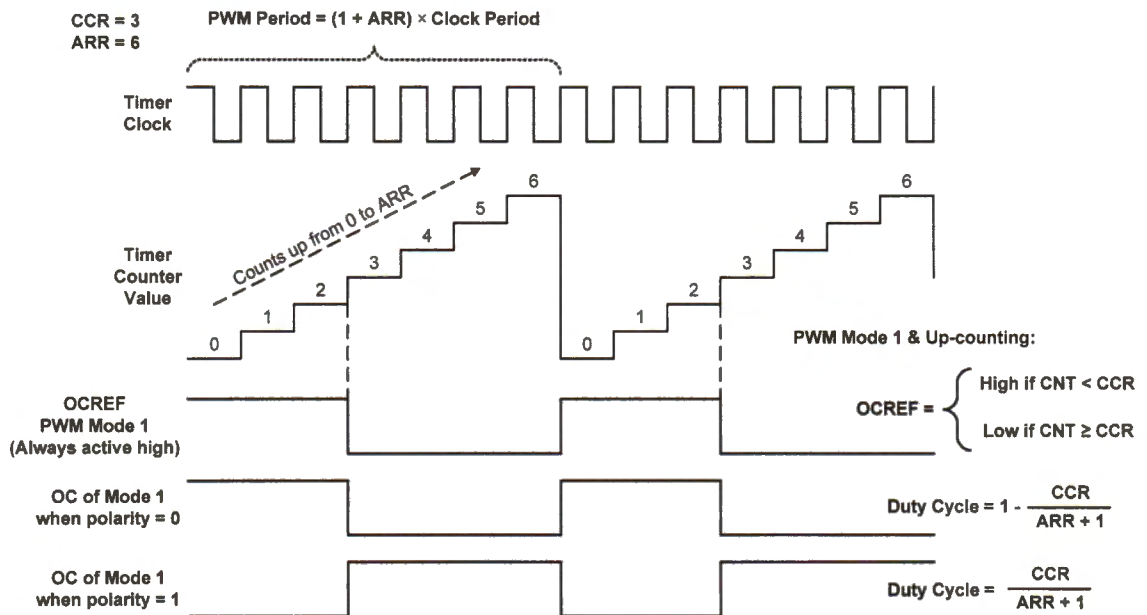


Figure 15-12. PWM outputs in Mode 1, up-counting, and different polarity bits

Similarly, as shown in Figure 15-13, depending on the PWM mode and the output polarity, the PWM duty cycle for center-counting is

$$\text{Duty Cycle} = \begin{cases} \frac{CCR}{ARR} \\ 1 - \frac{CCR}{ARR} \end{cases}$$

If the center-aligned counting mode, when register CCR is 0 or ARR, the timer reference output OCREF is 1 or 0, depending on the PWM mode.

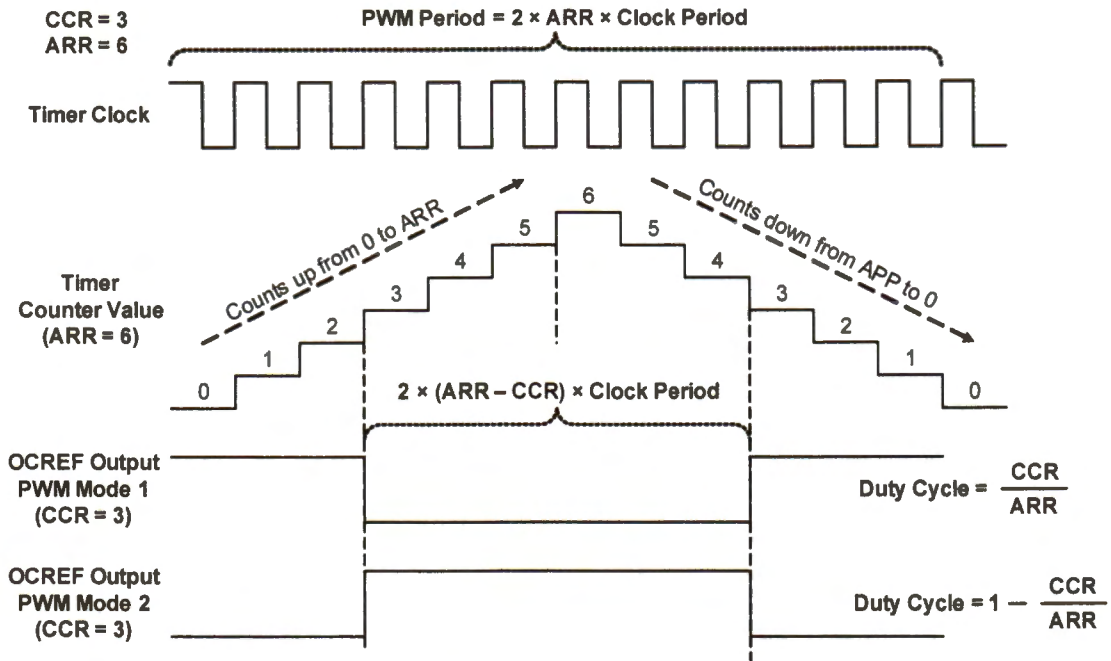


Figure 15-13. PWM outputs in center-aligned counting

15.3.1 PWM Alignment

A timer typically has multiple channels, as shown in Figure 15-4. Each channel can generate a PWM output signal. All channels share the timer counter and the ARR register. Thus, all PWM signals produced by the same timer have the same period. However, their duty cycle can be different because each channel has its own CCR register.

All PWM outputs of the same timer has the same period.

For a timer with up- or down-counting, its PWM signals are *edge-aligned*.

- In the example, given in Figure 15-14, all rising edges of the PWM signals are aligned with the overflow events of the timer counter. These PWM signals are also called *left-edge aligned* (or *rising-edge aligned*, or *leading-edge aligned*) because all pulses are turned on at the beginning of each PWM period.
- If the polarity bit is changed to 1, the PWM outputs of these three channels become *right-edge aligned*, or called *falling-edge aligned*, or *trailing-edge aligned* in which all pulses are turned off at the end of each PWM period.

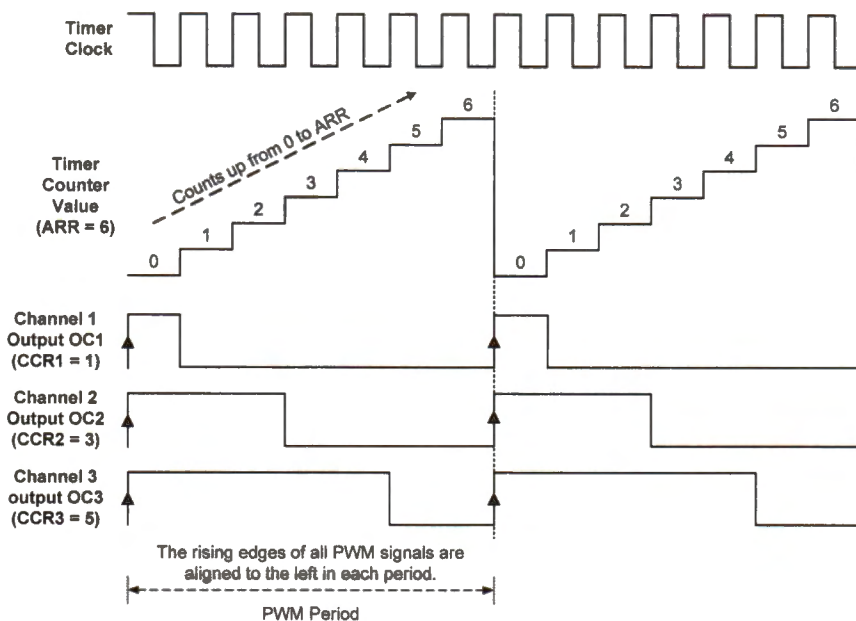


Figure 15-14. Rising edge aligned PWM outputs from multiple channels (up-counting, PWM mode 1, polarity bit = 0, Prescaler = 0).

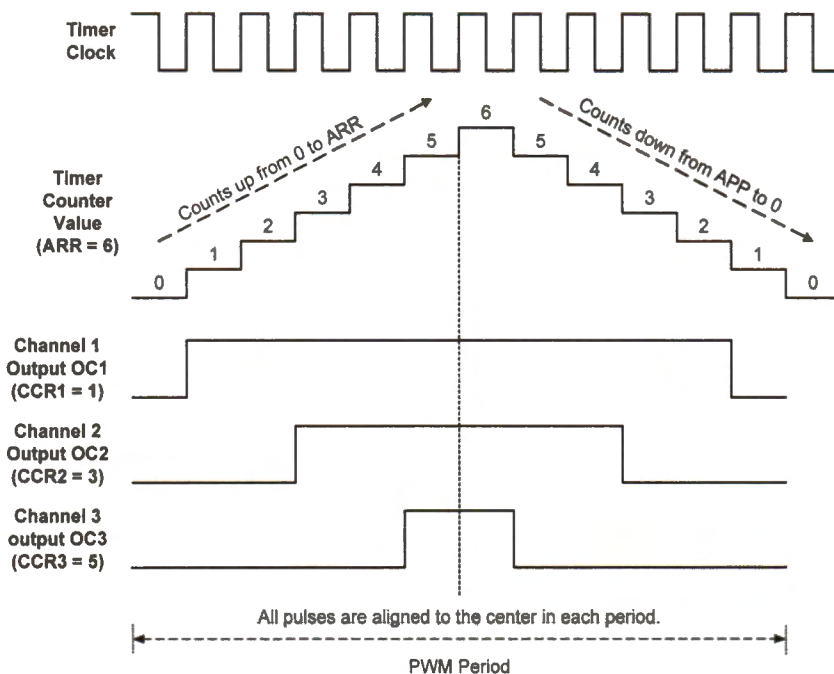


Figure 15-15. Center-aligned PWM outputs from multiple channels (center-counting, PWM mode 2, polarity bit = 0, Prescaler = 0)

For a timer with center-counting, the PWM signals of all associated channels are *center-aligned*, as shown in Figure 15-15. Both the rising edge and falling edge of each pulse are modulated. All pulses are centered in each PWM period. Center-aligned PWM signals are often preferred to edge-aligned in electronic circuits, such as motor control and power converters. Center-aligned PWM signals are symmetrical and thus have fewer harmonics, reducing noise interference and power consumption.

15.3.2 PWM Programming Flowchart

This section will show how software programs a timer to generate a PWM output signal to increase or decrease the brightness of an LED gradually. We will use the green LED on STM32L4 as an example (see Figure 15-20).

All alternate functions that pin PE 8 supports are listed in Table 15-2. Pin PE 8 will be the complementary output of channel 1 of timer 1 (TIM1_CH1N) if software sets its GPIO mode to alternate function (AF) and its AF value to 1. In sum, pin PE 8 is physically connected to the external LED, and software needs to make PE 8 function as TIM1_CH1N internally.

On STM32L4, the clock that drives the processor core and its peripherals, such as timers, has a frequency of 4 MHz by default. In this example, the prescaler factor is set as 39, thus the frequency at which the counter increments is

$$f_{CK_CNT} = \frac{f_{CL_PSC}}{Prescaler + 1} = \frac{4MHz}{39 + 1} = 100\text{ KHz}$$

Suppose the refresh rate of the LED is 100 Hz. Putting it differently, the LED is turned off and then turned on in each cycle. There are 100 such cycles each second. Normally the human eye cannot detect flicker at a refresh rate higher than 100 Hz. Thus, we will see a steady and continuously source of light even though the LED is switched on and off 100 times per second.

For a given clock frequency, ARR determines the PWM period. Assume the counting mode is up-counting, and the LED refresh rate is 100 Hz. Thus, the PWM period should be

$$PWM\ Period = \frac{1}{LED\ Refresh\ Rate} = \frac{1}{100\ Hz} = 0.01\ second$$

At the same time, we have

$$PWM\ Period = (1 + ARR) \times \frac{1}{f_{CK_CNT}}$$

Thus, we have

$$ARR = PWM\ Period \times f_{CK_CNT} - 1 = 0.01 \times 100 \times 10^3 - 1 = 999$$

When ARR is kept at a fixed value, CCR controls the duty cycle. Assume timer 1 uses PWM mode 1, in which the reference output (OCREF) is high for up-counting if the timer counter is less than CCR, and otherwise, OCREF is low.

Figure 15-16 shows that as CCR increases gradually, the duty cycle of OCREF also increases.

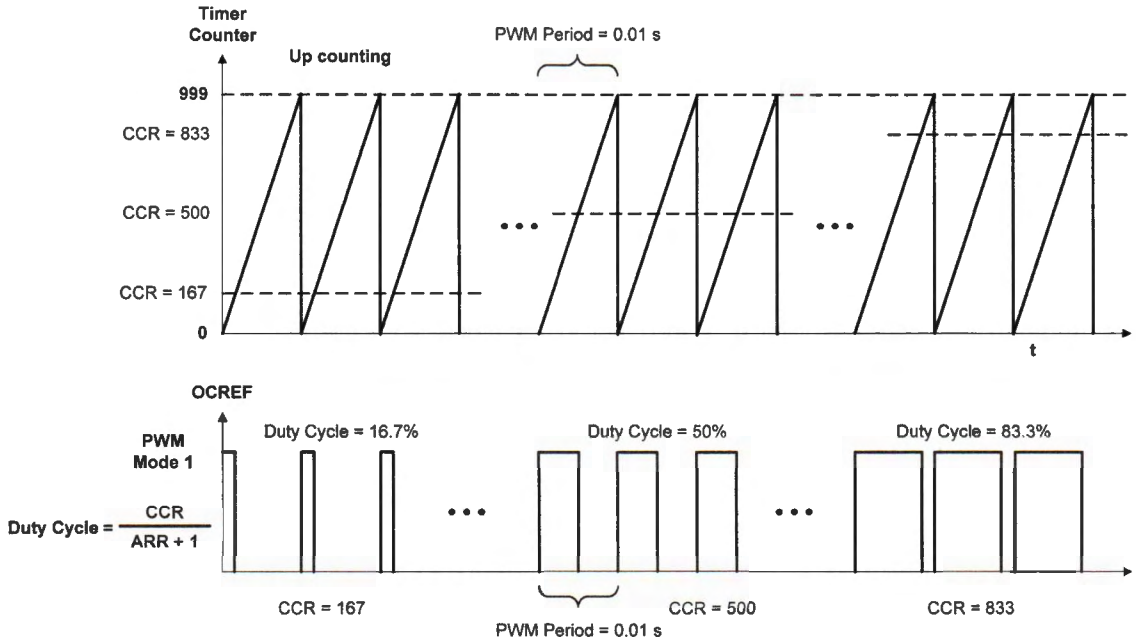


Figure 15-16. CCR controls the duty cycle of PWM output if ARR is fixed.

In this example, the complementary output of time 1 channel 1 (TIM1_CH1N) drives the LED. As discussed previously, the complementary output (OCN) of a channel is

$$\text{OCN} = \text{OCREF} + \text{Polarity bit for OCN}$$

If we use CCR to represent the LED brightness, we need to set the polarity bit (CCNP) for the complement output to 0. In other words, OCN waveform is the same as OCREF.

Figure 15-17 shows the program flow chart. The variable *direction* is either 1 or -1. The LED gets brighter and dimmer alternatively. A short delay is added between two consecutive updates to CCR to adjust the speed of brightness change.

In this example, software does not program the reset and clock controller (RCC). Accordingly, the processor uses the default clock settings. By default, the system clock is derived from the internal multi-speed internal RC oscillator (MSI). While MSI offers clocks signal with multiple clock frequencies ranging from 100 KHz to 49 MHz on STM32L4, its frequency is 4 MHz after reset.

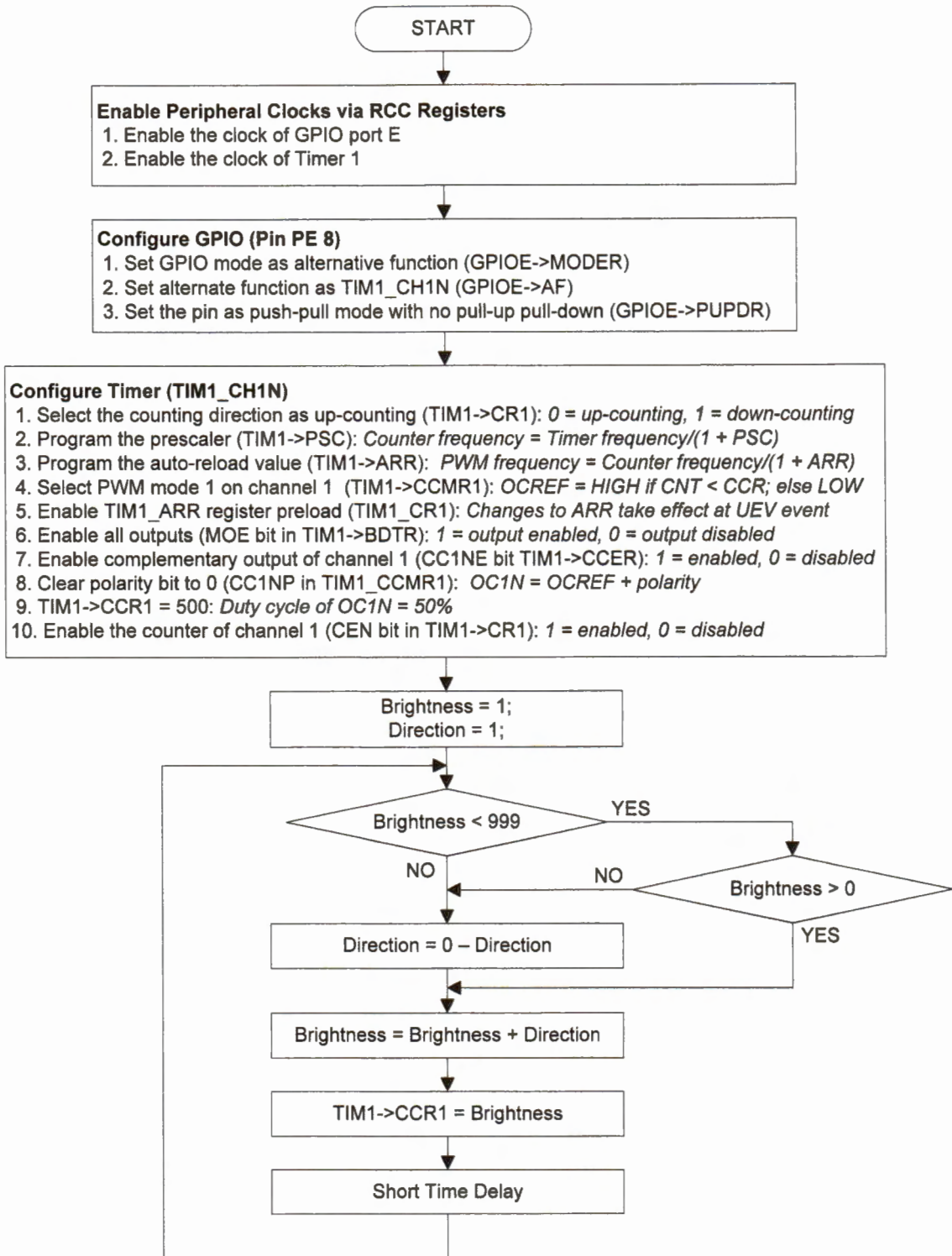


Figure 15-17. Flowchart for dimming an LED via PWM output of a timer

Example 15-4 initializes channel 1 of timer 1 to generate a PWM signal. The counting direction is up-counting. The PWM output is in mode 1.

- Each channel may have two outputs: the main output and the complementary output. Both can be enabled and disabled independently. Typically, these two outputs are connected to different GPIO pins on the processor chip.
- PE 8 is connected internally to the complementary output TIM1_CH1N, instead of the main output TIM1_CH1.

```
void TIM1_Init() {

    // Enable TIMER 1 clock
    RCC->APB2ENR |= RCC_APB2ENR_TIM1EN;

    // Counting direction: 0 = Up-counting, 1 = Down-counting
    TIM1->CR1 &= ~TIM_CR1_DIR; // Select up-counting

    // Prescaler, slow down the input clock by a factor of (1 + prescaler)
    TIM1->PSC = 39;    // 4 MHz / (1 + 39) = 100 KHz

    // Auto-reload
    TIM1->ARR = 999;    // PWM period = (999 + 1) * 1/100KHz = 0.01s

    // Clear output compare mode bits for channel 1
    TIM1->CCMR1 &= ~TIM_CCMR1_OC1M;

    // Select PWM Mode 1 output on channel 1 (OC1M = 110)
    TIM1->CCMR1 |= TIM_CCMR1_OC1M_1 | TIM_CCMR1_OC1M_2;

    // Output 1 preload enable
    TIM1->CCMR1 |= TIM_CCMR1_OC1PE;

    // Select output polarity: 0 = Active high, 1 = Active low
    TIM1->CCER &= ~TIM_CCER_CC1NP; // OC1N = OCREF + CC1NP

    // Enable complementary output of channel 1 (CH1N)
    TIM1->CCER |= TIM_CCER_CC1NE;

    // Main output enable (MOE): 0 = Disable, 1 = Enable
    TIM1->BDTR |= TIM_BDTR_MOE;

    // Output Compare Register for channel 1
    TIM1->CCR1 = 500;    // Initial duty cycle 50%

    // Enable counter
    TIM1->CR1 |= TIM_CR1_CEN;
}
```

Example 15-4. Initialization of Timer 1 to generate PWM output in C

Example 15-5 changes the duty cycle of the PWM output. The code does not configure the system clock and thus the default MSI 4 MHz is selected. The GPIO initialization code is not given in the demo code. Within the endless loop, the compare and capture register for channel 1 (CCR1) is increased or decreased gradually after a short delay.

```
void main(){

    int i;
    int brightness = 1;
    int stepSize = 1;

    // Pin PE.8 initialization:
    // AF mode, AF value = 1 (TIM1_CH1N), high-speed, push-pull
    ...

    while(1) {

        if ((brightness >= 999) || (brightness <= 0 ))
            stepSize = -stepSize;      // Reverse direction

        brightness += stepSize;        // Change brightness

        TIM1->CCR1 = brightness;       // Change duty cycle of channel 1 outputs

        for(i = 0; i < 1000; i++);    // Short delay
    }
}
```

Example 15-5. Example program to change the brightness.

15.4 Input Capture

As presented previously, a timer can be used for triggering an output at a specified time to general output signals (PWM output, comparator output). This section discusses the usage of timers as input capture.

Input capture is to find the time span between two rising or falling transitions in an internal or external signal. As shown in Figure 15-18, a capture occurs on (1) either rising or falling edges, (2) only falling edges, or (3) only rising edges.

When the desired transition is detected, the timer hardware automatically captures this time instant by copying the value of the free-run counter (CNT) to the compare and capture register (CCR). At the same time, the timer hardware generates an interrupt or DMA request and sets the CCIF flag in the status register (SR).

The difference between two consecutive transitions measures an elapsed time span, as shown in Figure 14-19.

- If the input signal is periodic, the difference of the counters captured at two consecutive rising edges or at two consecutive falling edges measures the period of the waveform.
- Similarly, the difference between a rising edge and a falling edge measures the pulse width.

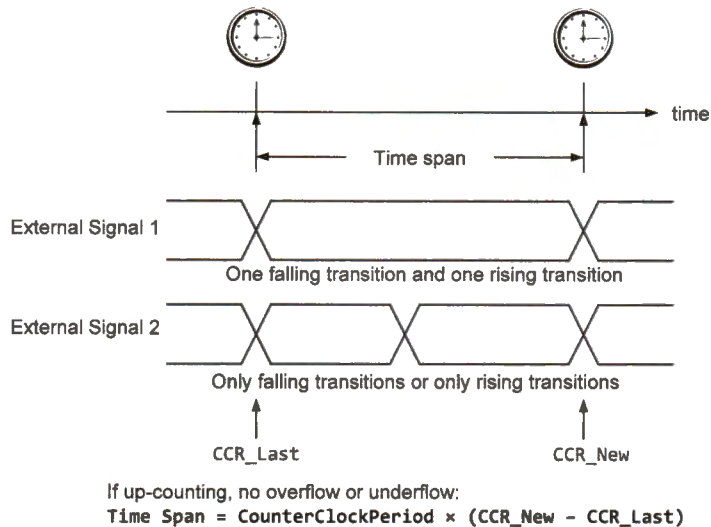


Figure 15-18. Input capture measures the time span between two events.

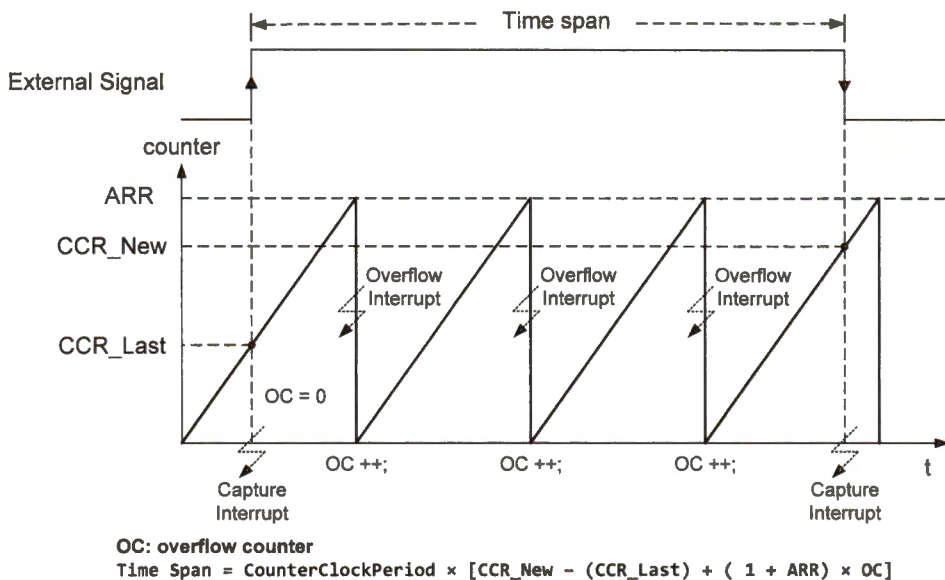


Figure 15-19. Measuring the pulse width if the timer is up-counting. A variable OC (overflow counter) is used when measuring a long time span.

We can calculate the pulse width, the period, and the duty cycle of the input waveform by using two or three consecutive captures. As shown in Figure 15-19, when measuring a time span longer than the counting period, software must consider the overflow and underflow events of the free-running counter.

Each input channel has a configurable input source, a digital input filter, and an edge detector.

- The input source of the capture module has a few options. The capture signal can be an external signal applied to different timer channels (*i.e.*, on different GPIO pins) or an internal trigger signal made by other timers.
- The edge detector can be programmed to detect only falling edges, only rising edges or both.
- The filter specifies the number of events needed to validate a transition on the input. If we need to capture each valid transition, software should disable the external trigger filtering by setting the external trigger filter (ETF) to zero.

The digital input filter removes noise pulses in an input signal. For example, if the input signal of a push button takes 10 internal clock cycles to become stable, then we can make the filter duration last longer than 10 clock cycles. We can validate the transition by repeatedly sampling the inputs.

- If a sequence of consecutive samples remain unchanged and stay at the same level, then this level is considered as a stable input.
- If a noise spike occurs causing the input to change during any of the consecutive sampling points, the input is unstable and can be ignored, thus filtering out noisy transitions.

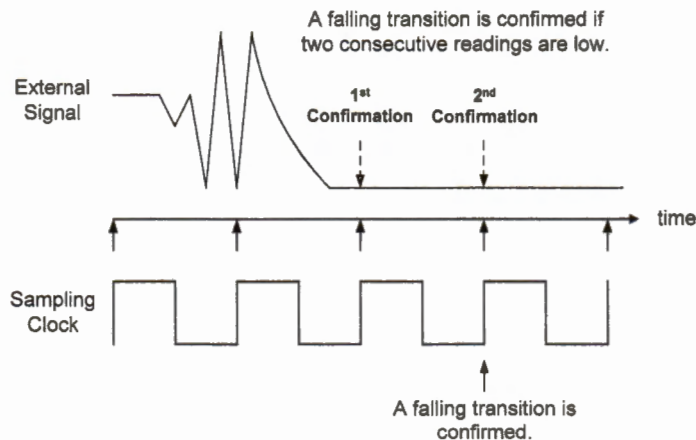


Figure 15-20. An example of filtering noise in the external signal. Two consistent readings are required to confirm a valid transition.

The sampling frequency of the filter is configurable. The number of consistent consecutive readings to validate a transition ranges from 0 to 8. If it is zero, then the noise filtering is disabled. Figure 15-20 shows an example in which the number of consistent readings is set to 2.

Software can select internal clocks, external clocks, or internal trigger input (such as another timer) as the clock of the 16-bit free-running counter. The clock frequency can be divided by a 16-bit clock prescaler (PSC) to reduce the clock frequency for the free-running counter.

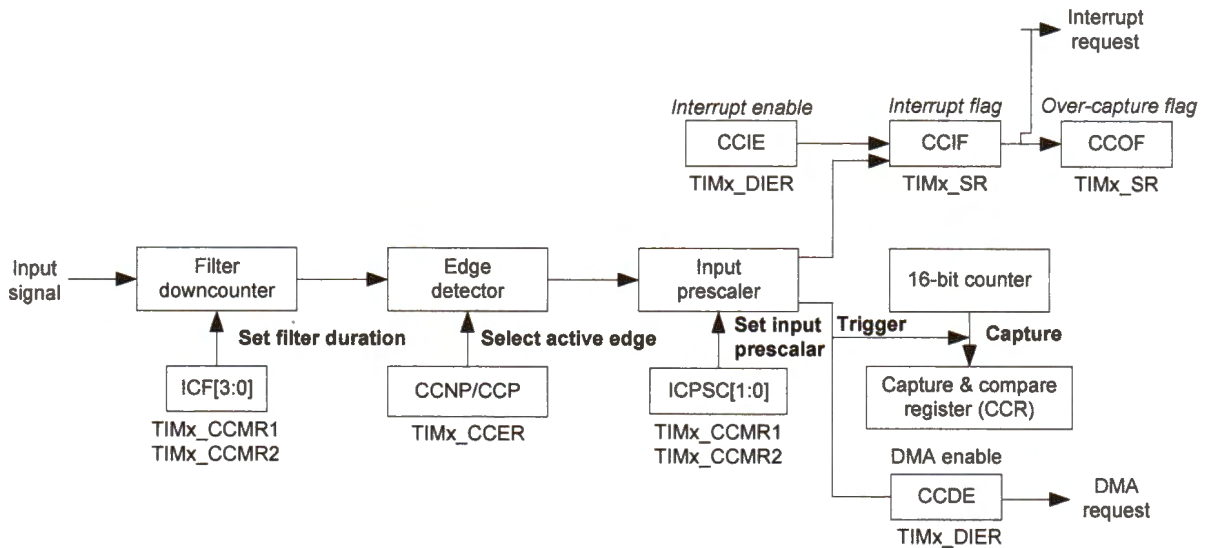


Figure 15-21. Diagram of input capture

Figure 15-21 shows the timer diagram for input capture. When the number of transitions on an input channel reaches the threshold defined by the *input capture prescaler* bits (ICPSC) in the capture/compare mode register (CCMR), a capture occurs, and the processor automatically performs the following operations.

- It latches the value of the free-running counter (CNT) to the capture/compare register (CCR) corresponding to that channel. There is a 16-bit capture/compare for each capture channel (CCR1 for channel 1, CCR2 for channel 2, CCR3 for channel 3, and CCR4 for channel 4).
- In the status register (SR), the capture/compare interrupt flag (CCIF) corresponding to that channel is set by hardware on a capture. If the CCIF flag has already been set, then hardware sets the corresponding capture/compare over-capture flag (CCOF) in the status register (SR). Software clears the CCIF flag by writing it to 0, or hardware automatically clears it if software reads the corresponding CCR register. Only software can clear the CCOF flag.

- If software enables the interrupt by setting the capture/compare interrupt enable bit (CCIE) in the DMA interrupt enable register (DIER), the timer hardware generates an interrupt.
- If software enables DMA by setting the capture/compare DMA request enable bit (CCDE) of the DIER register, the timer hardware generates a DMA request.

15.4.1 Configuring Input Capture

In the following, we use the channel 1 of timer 4 as an example to illustrate the basic procedure for capturing a rising edge transition of an external signal. Figure 15-23 shows the flowchart of setting the GPIO pin PB.6 as input capture.

1. **Select the active input.** Each timer supports four capture/compare channels in the STM32L processor, and software can configure each channel as a compare output or a capture input connected to one of the timer's external input or trigger sources. The CC1S[1:0] bits in the CCMR1 register configures the direction of channel 1 (input or output) and the input source of channel 1.

The usage of CC1S[1:0] bits are defined as follows:

- 00: Channel 1 is set as output.
- 01: Channel 1 is set as input, and it is mapped to timer input 1 (TI1).
- 10: Channel 1 is set as input, and it is mapped to timer input 2 (TI2).
- 11: Channel 1 is set as input, and it is mapped to TRC. The trigger output of another timer is used as the input.

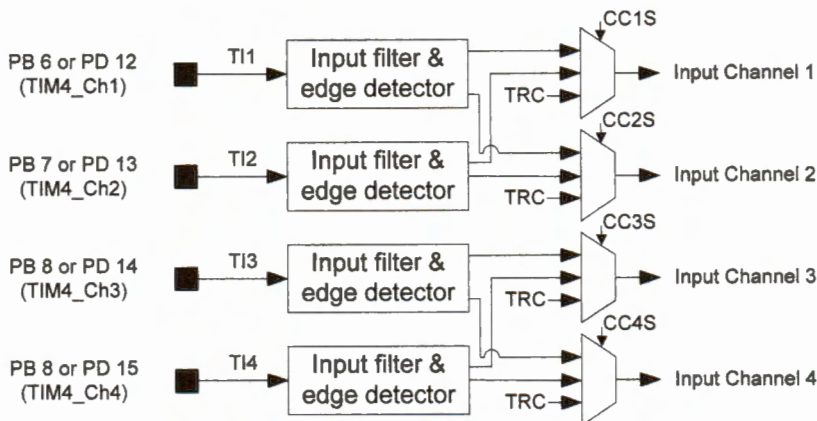


Figure 15-22. Selecting an input source for each capture channel

To link to the timer input 1 (TI1), we need to set CC1S bits to 01. Note, if CC1S bits are 00, then the channel is configured as a compare output. When a channel is set up as capture input, its corresponding CCR register becomes read-only.

2. **Set the input filter.** The IC1F bits in the CCMR1 register define the frequency used to sample TI1 and the length of digital filtering applied to TI1. The digital filtering is to check whether a number of consecutive readings of an input remain the same. If yes, the input is stable, and the input transition is valid. For example,
 - If IC1F is 0000, the digital filtering is disabled.
 - If IC1F is 0010, the sampling frequency is the counter frequency, and if four consecutive samplings remain the same, then the transition is valid.
3. **Set the active edge.** The CC1P bit and CC1NP bit in the CCER register collectively determine the edge of an active transition on the TI1 channel.
 - If CC1NP = 0 and CC1P = 0, then the edge detection is configured to capture only rising edges.
 - If CC1NP = 0 and CC1P = 1, only falling edges are captured.
 - If CC1NP = 1 and CC1P = 1, then both falling and rising edges are captured.
4. **Set the input prescaler.** If we want to capture the event each time an edge is detected in channel 1, the prescaler, defined by the IC1PSC bits in the CCMR1 register, should be reset to 0.
 - When IC1PSC is 01, 10, and 11, the input capture is then performed once every 2, 4, and 8 events, respectively.
 - Depending on applications' needs, such as noise filtering, we might want to wait for multiple valid transitions before the counter is latched.
5. **Enable the input capture.** The input capture of channel 1 can be enabled by setting the CC1E bit in the CCER register.
6. **Enable interrupt and DMA if needed.** The related interrupt request is enabled by setting the CC1E bit in the DIER register.
 - The DMA request is enabled by setting the CC1DE bit in the DIER register.
 - The TIE bit enables trigger interrupt, and the UIE bit enables the update interrupt.
7. **Enable the timer counter.** Sets the counter enable bit (CEN) in the CR1 register.

Input filtering is to
reduces noise.

Input prescaler, also
called *frequency divider*,
is to reduce the frequency
signal of input signals.

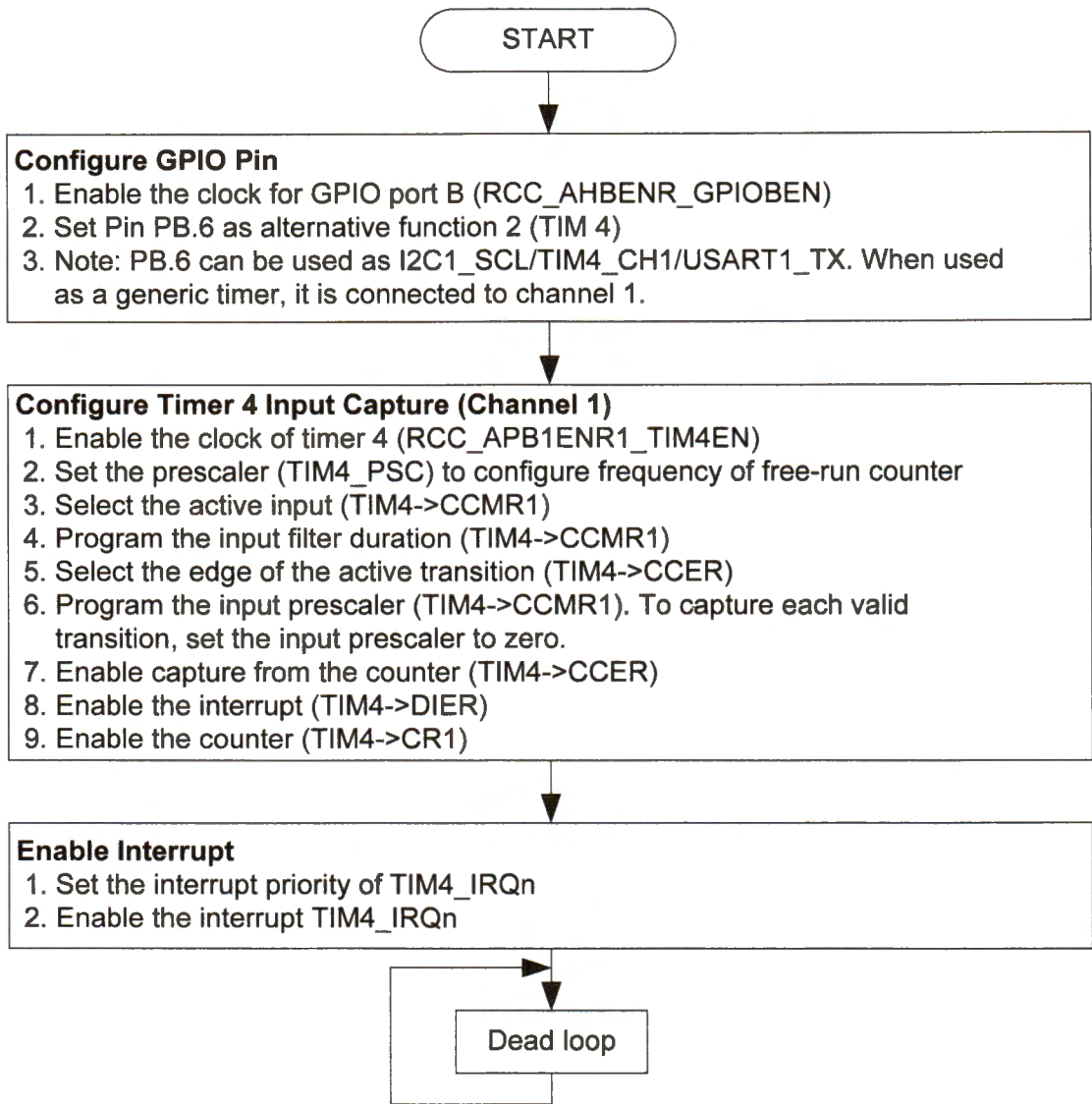


Figure 15-23. Flowchart to set the GPIO pin PB.6 as input capture

Example 15-6 shows the implementation of the timer interrupt service routine, which calculates the pulse width. It is assumed that the timer is up-counting and captures are made on both rising and falling edges. Assuming the input signal is low initially, it uses a global variable to track the signal polarity. The interrupt handler flips the signal polarity. The pulse duration is calculated only when the signal is low.

This example code does not take care of the counter overflow and, therefore, the time interval may be negative occasionally. A more robust code should check the update interrupt flag (TIM_SR_UIF) of the timer status register (TIM4->SR). If the update event

occurs (*i.e.*, overflow during up-counting or underflow during down-counting), the time interval should be calculated differently.

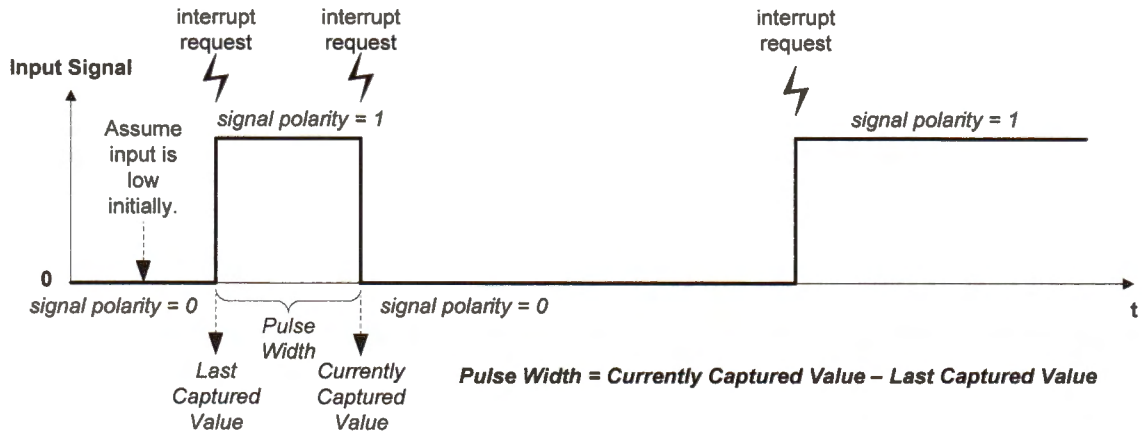


Figure 15-24. Calculating the pulse width by using input capture

```
volatile uint32_t pulse_width = 0;
volatile uint32_t last_captured = 0;
volatile uint32_t signal_polarity= 0; // Assume input is low initially

void TIM4_IRQHandler() {
    uint32_t current_captured;

    if((TIM4->SR & TIM_SR_CC1IF) != 0) { // Check interrupt flag is set
        // Reading CCR1 clears CC1IF interrupt flag
        current_captured = TIM4->CCR1;

        // Toggle the polarity flag;
        signal_polarity= 1 - signal_polarity;

        if (signal_polarity == 0){ // Calculate only when the current input is Low
            pulse_width = current_captured - last_captured; // Assume up-counting
        }

        last_captured = current_captured;
    }

    if((TIM4->SR & TIM_SR_UIF) != 0) { // Check if overflow has taken place
        TIM4->SR &= ~TIM_SR_UIF; // Clear UIF flag to prevent re-entering
    }
}
```

Example 15-6. Timer interrupt handler in C. Assume the timer is up-counting.

Table 15-4 shows the configuration of pin PB 6 as timer function. The pin is set as no pull-up and no pull-down, and alternate function mode.

GPIO Pin	Connection	Mode	AF	Pull-up/Pull-down
Port B pin 6 (PB 6)	LED	AF	TIM4_CH1	No pull-up, No pull-down

Table 15-4. Connecting PB.6 to channel 1 of timer 4 internally

The following gives an example code of setting pin PB 6 (TIM4_CH1) as input capture. Register CCR stores the value of the counter after a transition detected. When a capture occurs, CCIF flag is set in the status register SR.

```
#include <stdint.h>
#include "stm32l1xx.h"

int main(void) {
    // Enable GPIO Port B clock and configure GPIO port B pin 6 as
    // AF function 2 (i.e., TIM4_CH1) with no pull-up/pull-down
    ...

    // Enable the clock to timer 4
    RCC->APB1ENR1 |= RCC_APB1ENR1_TIM4EN;

    // Set up an appropriate prescaler to slow down the timer's input clock
    TIM4->PSC = 127;

    // Set Auto-reload value to maximum value
    TIM4->ARR = 0xFFFF; // Maximum 16-bit value

    // Set the direction of channel 1 as input, and select the active input
    TIM4->CCMR1 &= ~TIM_CCMR1_CC1S; // Clear capture/compare 1 selection bits
    TIM4->CCMR1 |= TIM_CCMR1_CC1S_0; // CC1S[1:0] for channel 1:
        // 00 = output
        // 01 = input, CC1 is mapped on timer input 1
        // 10 = input, CC1 is mapped on timer input 2
        // 11 = input, CC1 is mapped on slave timer

    // Program the input filter duration: Disable digital filtering by clearing
    // IC1F[3:0] bits because we want to capture every event
    TIM4->CCMR1 &= ~TIM_CCMR1_IC1F; // No filtering

    // Set the active transition as both rising and falling edges
    // CC1NP:CC1P bits: 00 = rising edge 01 = falling edge
    // 10 = reserved 11 = both edges
    TIM4->CCER |= TIM_CCER_CC1P | TIM_CCER_CC1NP; // Both edges generate interrupts

    // Program the input prescaler: Clear prescaler to capture each transition
    TIM4->CCMR1 &= ~(TIM_CCMR1_IC1PSC);
```



```

// Enable capture for channel 1
// CC1E: 0 = disabled, 1 = enabled
TIM4->CCER |= TIM_CCER_CC1E;

// Allow channel 1 of timer 4 to generate interrupts
TIM4->DIER |= TIM_DIER_CC1IE;

// Allow channel 1 of timer 4 to generate DMA requests
TIM4->DIER |= TIM_DIER_CC1DE; // Optional. Required if DMA is used

// Enable the timer counter
TIM4->CR1 |= TIM_CR1_CEN;

// Set priority of timer 4 interrupt to 0 (highest urgency)
NVIC_SetPriority(TIM4_IRQn, 0);

// Enable timer 4 interrupt in the interrupt controller (NVIC)
NVIC_EnableIRQ(TIM4_IRQn);

while(1);
}

```

Example 15-7. Implementation of the initialization of a timer for input capture

15.4.2 Input Capture in Slave Mode with Reset

The interrupt handler in Example 15-6 can be simplified to the following if software sets timer 4 channel 1 as the slave mode with automatic reset.

```

volatile uint32_t pulse_width = 0;

void TIM4_IRQHandler(void) {

    if((TIM4->SR & TIM_SR_UIF) != 0) // Check if overflow has taken place
        TIM4->SR &= ~TIM_SR_UIF;    // Clear UIF Flag to prevent re-entering

    if((TIM4->SR & TIM_SR_CC1IF) != 0) // Check interrupt flag is set
        pulse_width = TIM4->CCR1;    // Reading CCR1 clears CC1IF flag
}

```

Example 15-8. Interrupt handler if timer is in slave mode

Figure 15-25 illustrates the basic concepts. The timer uses each rising edge to reset the timer counter, and each falling edge to make a capture and generate a timer interrupt. For up-counting, the timer counter is re-initialized to zero upon a reset. Example 15-9 is the additional configuration that selects the slave mode with reset. This code should be added before the channel is enabled, *i.e.*, before software sets the CC1E bit in the CCER register. In this mode, a rising edge of the input signal reinitializes the counter, and a falling edge generates a timer interrupt. The pulse width is the CCR value captured.

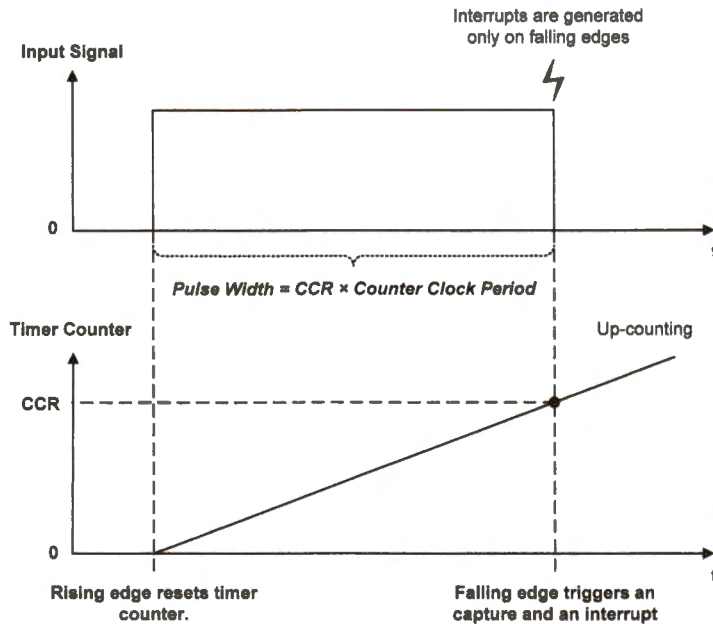


Figure 15-25. Input capture when the timer is set as slave mode with automatic reset

```
// Select falling edges as the active transition to generate interrupts
// CC1NP:CC1P bits: 00 = rising edge  01 = falling edge
//                  10 = reserved      11 = both edges
TIM4->CCER &= ~(TIM_CCER_CC1P | TIM_CCER_CC1NP); // Clear polarity bits
TIM4->CCER |= TIM_CCER_CC1P; // Only falling edges generate interrupts

// Select the input trigger source
TIM4->SMCR &= ~TIM_SMCR_TS; // Clear the trigger selection bits
TIM4->SMCR |= 4UL << 4; // 100: TI1 Edge Detector (TI1F_ED)

// Select slave mode with reset
TIM4->SMCR &= ~TIM_SMCR_SMS; // Clear slave mode bits
TIM4->SMCR |= 4; // 0100 Slave mode with reset
```

Example 15-9. Setting Timer 1 as the slave mode with reset

Figure 15-26 shows the selection of the input signal to the slave mode controller and the capture signal to the capture module for channel 1.

- The input signal input (TGRI) to the slave mode controller can be selected from TI1F_ED, TI1FP1, TI2FP2, ETF, and various ITR from other timers. Depending on SMS bits in the SMCR register, the input signal can be used to trigger the reset of the timer counter (reset mode), stop and start the time counter based on polarity (gate mode) or start the counter on rising edges of TGRI (trigger mode). In Example 15-9, TGRI is used to reset the time counter. Specifically, it selects TI1F_ED as the trigger source. As a result, the counter is reset on both rising and falling edges of TI1F.

- The capture signal to the capture module can be TI1FP1, TI2FP2 or TRC. Example 15-9 selects TI1FP1 as the capture signal. Also, a capture is made on each falling edge of TI1F.

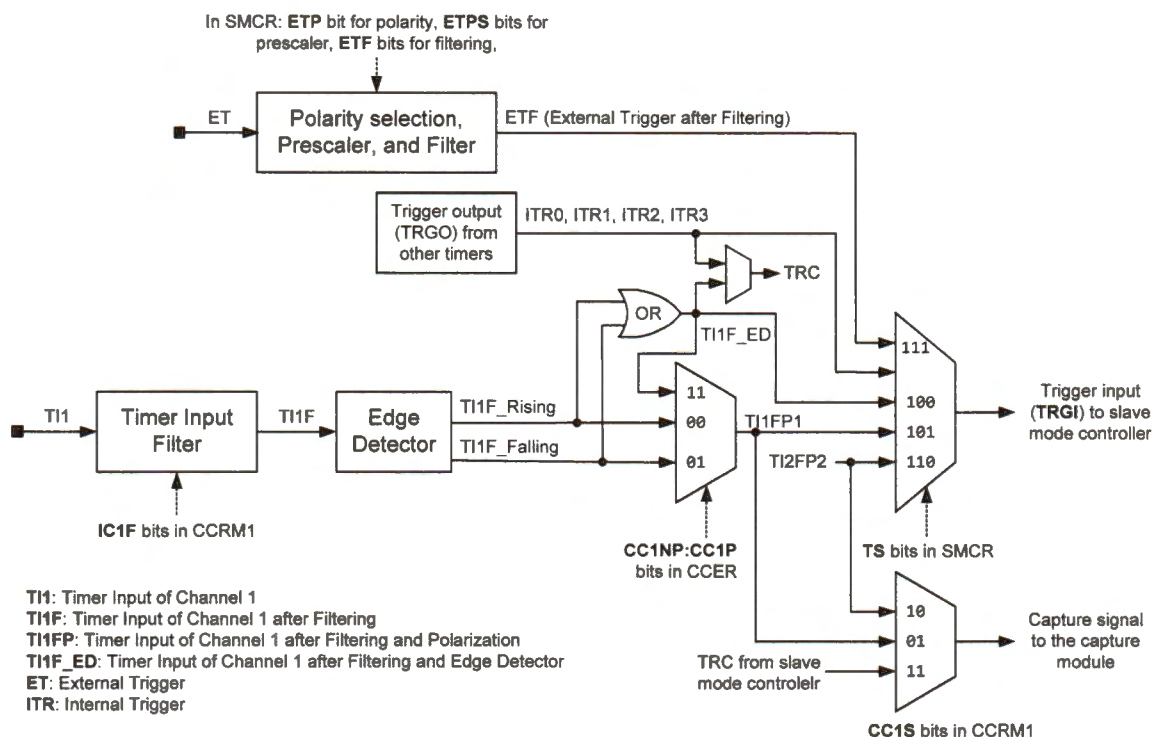


Figure 15-26. Internal trigger and capture signals for channel 1

Figure 15-27 shows an example time diagram of measuring the pulse width. It is assumed that no filtering has been applied to the input signal (TI1).

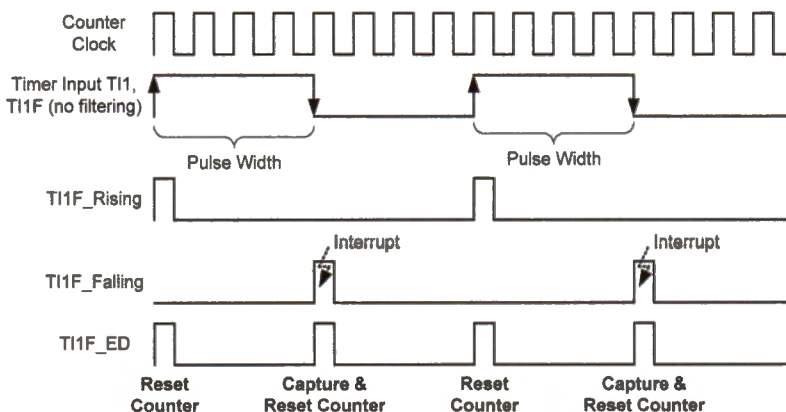


Figure 15-27. Counter reset events and capture events in slave mode with reset

15.4.3 Interfacing to Ultrasonic Distance Sensor

An ultrasonic distance sensor has one transmitter and one receiver. The transmitter generates short bursts of high-frequency ultrasonic waves. The receiver detects any wave reflected back from the target, as shown in Figure 15-28.

Without contacting with the target physically, it measures the difference in time between sending waves and receiving reflected waves. The distance is then calculated as follows:

$$\text{Distance} = \frac{\text{Round Trip Time} \times \text{Speed of Sound}}{2}$$

One example application is an automatic door opener, which opens a door when a person approaches. Compared with optical distance sensors, ultrasonic distance sensors are low cost but less accurate.

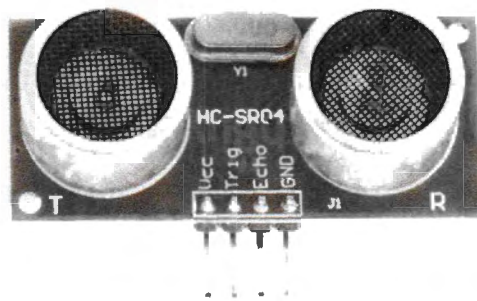


Figure 15-28. Ultrasonic distance sensor of HC-SR 04
(VCC = +5V, Trig = Trigger input to sensor, Echo = Echo time output of sensor)

To start a distance measurement, the processor should send a high pulse signal ($\geq 3.2\text{V}$) with a width of $10\text{ }\mu\text{s}$ to the trig pin. The ultrasonic transmitter then sends out 8 cycles of 40-KHz ultrasonic waves (*i.e.*, for $200\text{ }\mu\text{s}$), which is greater than the upper limit of human hearing range (typically 20 KHz).

When the ultrasonic receiver detects any waves reflected within a predefined time window, it generates a high pulse (5V) on the echo pin. The pulse width is linearly proportional to the distance of the nearest objects.

Specifically, if the width of the pulse on the echo pin is in microseconds (μs), the distance, measured in centimeters, is calculated as follows:

$$\text{Distance} = \frac{\text{Pulse Width } (\mu\text{s})}{58} \text{ cm}$$

or

$$\text{Distance} = \frac{\text{Pulse Width } (\mu\text{s})}{148} \text{ inch}$$

The sensor can measure a distance between 2 *cm* and 400 *cm*, with a resolution of 0.3 *cm*, and the corresponding echo pulse width is between 150 μs and 25 *ms*. When the sensor detects no object, the echo pulse width is 38 *ms*.

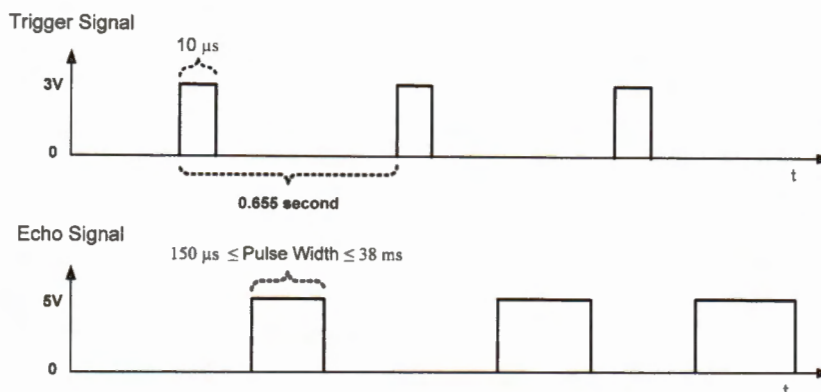


Figure 15-29. Trigger and echo signals

Suppose we want to measure the distance every 0.655 seconds, as shown in Figure 15-29 and Figure 15-30. We can use two timers to interface the ultrasonic distance sensor.

- The first timer, which is set as PWM output mode 1 and connected to the trigger pin of the sensor, generates a pulse signal with a width of 10 μs .
- The second timer, which is set as input capture and connected to the echo pin of the sensor, measures the pulse width of the echo signal. The second timer should be able to detect both the rising and falling edge of the echo signal.

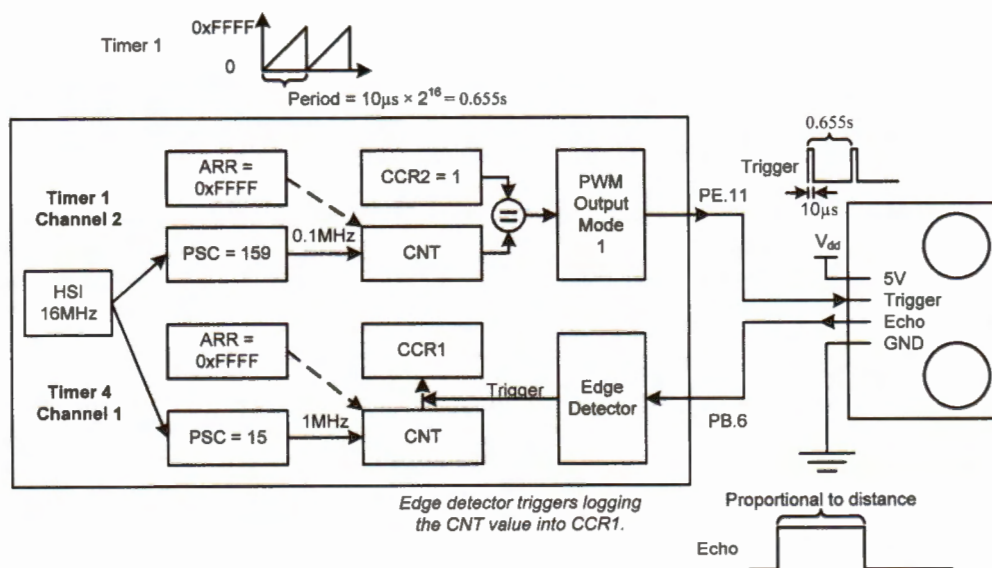


Figure 15-30. Setup of timers for interfacing the ultrasonic distance sensor.

As shown Figure 15-30, we use Timer 1 Channel 2 to drive the trigger pin. Suppose 16 MHz HSI is selected to drive the timers. The following shows the calculation of the period and pulse width of sensor trigger signal.

$$\text{Timer 1 Counter Clock} = \frac{\text{Timer 1 Input Clock}}{1 + \text{Prescaler}} = \frac{16 \text{ MHz}}{1 + 159} = 0.1 \text{ MHz}$$

$$\text{PWM Period} = (1 + \text{ARR}) \times \text{Counter Clock Period} = (1 + 0xFFFF) \frac{1}{0.1 \text{ MHz}} = 0.655 \text{ s}$$

$$\text{PWM pulse width} = \text{CCR} \times \text{Counter Clock Period} = 1 \times \frac{1}{0.1 \text{ MHz}} = 10 \mu\text{s}$$

The distance measured is only in one direction with limited angular resolution. We can use multiple ultrasonic sensors to detect objects in several directions. Also, it is possible that an object reflects ultrasonic waves away if the target surface is oriented at unfavorable angles, resulting in the object being undetected. Additionally, objects with a soft or irregular surface might not reflect enough ultrasonic waves back, and accordingly, ultrasonic sensors fail to detect them. Moreover, sounds travel slower in colder air, and thus the program should perform some calibration to achieve better accuracy.

The following assembly code selects the 16MHz HSI clock as the system clock.

```
; Select HSI (16 MHz, 1% accuracy) as the system clock.
```

```
HSI_init PROC
```

```
    EXPORT HSI_init
```

```
    ; Turn on HSI oscillator
```

```
    LDR r0, =RCC_BASE
```

```
    LDR r1, [r0, #RCC_CR]
```

```
    ORR r1, r1, #RCC_CR_HSION
```

```
    STR r1, [r0, #RCC_CR]
```

```
    ; Select HSI as system clock
```

```
    LDR r1, [r0, #RCC_CFGR]
```

```
    BIC r1, r1, #RCC_CFGR_SW
```

```
    ORR r1, r1, #RCC_CFGR_SW_HSI
```

```
    STR r1, [r0, #RCC_CFGR]
```

```
    ; Wait for HSI stable
```

```
WaitHSI LDR r1, [r0, #RCC_CR]
```

```
    AND r1, r1, #RCC_CR_HSIRDY
```

```
    CMP r1, #0
```

```
    BEQ WaitHSI
```

```
    BX LR
```

```
ENDP
```

Example 15-10. Configuring high-speed internal clock (HSI)

Timer 4 uses the input capture to record the time instant when a rising or falling edge takes place in the echo signal, as shown in Figure 15-30. Software should set the update interrupt enable flag (UIE) and the capture event flag (CC1IE for channel 1) in the DMA/interrupt enable register (DIER) to allow these interrupts.

Timer 4 is set as counting upward. A counter overflow occurs when the 16-bit timer counter reaches $0xFFFF$. Because the counter increments every $1\mu s$, an overflow occurs every $2^{16} \times 1\mu s = 6.5\text{ ms}$ in this example. When an overflow occurs, the timer generates an interrupt and sets up the UIF flag bit in the interrupt status register (SR).

Even though the duration of the echo signal from the sensor is limited to 38 ms , it is possible that the counter can overflow when measuring the pulse width of the echo signal, as shown in Figure 15-31.

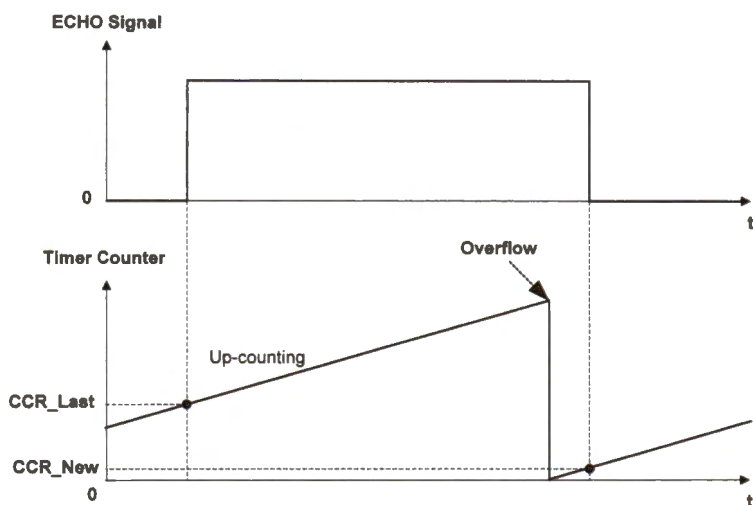


Figure 15-31. Overflow occurs between two capture events

Thus, we need a variable (named *overflow* in the following code) that counts the number of overflows within one pulse. The time span can be calculated as follows:

$$\text{Time Span } (\mu s) = (CCR_{New} - CCR_{Last}) + 65536 \times \text{Overflows}$$

in which CCR_{New} is currently captured value and CCR_{Last} is the last captured value. The variable *Overflows* should be reinitialized to zero after the time span is calculated. After calculating the time span, the program should use the current counter value to update the last counter value to prepare the next calculation.

Example 15-11 shows the assembly implementation of the interrupt handler for timer 4 (TIM4_IRQHandler). The three variables *timespan*, *lastCounter*, and *overflow* should be defined in the data area.

```

; These three variables should be defined in the data area
timespan      DCD    0    ; Pulse width
lastCounter   DCD    0    ; Timer counter value of last capture event
overflow      DCD    0    ; Counter the number of overflows

; The following should be in the code area
TIM4_IRQHandler PROC
    EXPORT      TIM4_IRQHandler

    PUSH {r4, r6, r10, lr}

    LDR r0, =TIM4_BASE      ; Pseudo instruction
    LDR r2, [r0, #TIM_SR]    ; Read status register
    AND r3, r2, #TIM_SR_UIF ; Check update event flag
    CBZ r3, check_CCFlag    ; Compare and branch on zero

    LDR r3, =overflow
    LDR r1, [r3]             ; Read overflow from memory
    ADD r1, r1, #1           ; Increment overflow counter
    STR r1, [r3]             ; Save overflow to memory

    BIC r2, r2, #TIM_SR_UIF ; Clear update event flag
    STR r2, [r0, #TIM_SR]    ; Update status register

check_CCFlag   AND r2, r2, #TIM_SR_CC1IF ; Check capture event flag
               CBZ r2, exit             ; Compare and branch on zero

    LDR r0, =TIM4_BASE      ; Load base memory address
    LDR r1, [r0, #TIM_CCR1] ; Read the new captured value

    LDR r2, =lastCounter
    LDR r0, [r2]             ; Load the last counter value
    STR r1, [r2]             ; Save the new counter value
    CBZ r0, clearOverflow    ; Compare and branch on zero

    LDR r3, =overflow
    LDR r4, [r3]             ; Load the overflow value
    LSL r4, r4, #16          ; Multiply by 2^16 (i.e., 65536)
    ADD r6, r1, r4
    SUB r10, r6, r0           ; r10 = timer counter difference
    LDR r2, =timespan
    STR r10, [r2]            ; Update timespan in memory

clearOverflow  MOV r0, #0
               LDR r3, =overflow
               STR r0, [r3]           ; clear overflow counter
exit          POP {r4, r6, r10, pc}
               ENDP

```

Example 15-11. Implementation of interrupt handler in assembly for measuring pulse width

In the above code, we use “LSL r4, r4, #16” instruction to replace multiplication with 65536 (i.e., 2^{16}). A shift instruction runs much faster than a multiplication instruction. Additionally, the *lastCounter* variable is initialized to 0.

The code above calculates the time span between a rising edge and a falling edge. We can improve the code by only calculating the time span from a rising edge to a falling edge, and ignore the time span from a falling edge to a rising edge.

One common mistake is that the interrupt handler does not clear the update event flag in the interrupt status register (SR). In the above code, the hardware sets up interrupt flags in SR.

- If the UIE bit in the DMA/interrupt enable register (DIER) is set, hardware sets the update flag when a counter overflow (or a counter underflow if the counter counts down) occurs.
- If the capture/compare interrupt enable (CCIE) bit is set in the DIER register, hardware sets the capture flag (CCIF) when a rising edge or a falling edge of an external signal is detected. Hardware also sets the over-capture flag (CCOF) if a new capture is made while the CCIF flag is set.

Interrupt handlers must clear interrupt flags in the status register to prevent the processor from re-entering the handler.

Hardware automatically clears the capture flag (CCIF) when software reads register (CCR). Therefore, *TIM4_IRQHandler* does not clear the CCIF flag explicitly.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0													
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0													
														CC6IF	CC5IF																SBIF	CC4OF	CC3OF	CC2OF	CC1OF	B2IF	B1F	TIF	COMIF	CC4IF	CC3IF	CC2IF	CC1IF	UIF

CCxIF: Interrupt flag on channel x

- For output on channel x, hardware sets this flag when the counter CNT matches register CCR.
- For input on channel x, hardware sets this flag when the counter CNT has been latched into register CCR.

CCxOF: Over-capture flag on channel x

- For input on channel x, hardware sets this flag when a new capture has been made while CCxIF was already set.

UIF: Update interrupt flag

- Hardware sets this flag when the counter CNT overflows during up-counting or the counter CNT under-flows during down-counting.

Figure 15-32. Timer status register (SR)

However, the interrupt handler must clear the update flag (UIF) in the status register (SR). If it were not cleared, the timer interrupt handler would repeatedly be called, which not only generates errors in calculating a time duration but also makes the processor have no time to run other codes with a lower priority.

15.5 Exercises

1. Suppose we want to use a timer to generate a PWM signal.
 - PWM has a duty cycle of 25%.
 - PWM has a fixed frequency of 320 Hz
 - The input clock to the timer is 32 MHz.
 - The timer uses PWM mode 2: the output is high if the counter is larger than or equal to the content of CCR.

How would you design the prescaler (PSC), the auto-reload register (ARR), and the compare and capture (CCR)? Show your calculation.

2. Use an oscilloscope to measure duty cycles. Timer input clock has 2.097 MHz.
 - a. What is the relationship between the timer input clock, the counter clock CK_CNT, the prescaler PSC, and the pulse period measured?
 - b. We want to keep TIM4_ARR fixed, but set TIM4_CCR1 to three different values, as listed below. How would you set up the ARR, PSC, and CCR register values? Calculate the duty measured and verify the correctness.
 - Case 1: $TIM4_CCR1 = 1/6 * (TIM4_ARR + 1)$
 - Case 2: $TIM4_CCR1 = 1/3 * (TIM4_ARR + 1)$
 - Case 3: $TIM4_CCR1 = 1/2 * (TIM4_ARR + 1)$
3. Suppose the HSE (high-speed external clock) of 16 MHz is selected as the clock of the timer. To generate a 1Hz square wave with a duty cycle of 50%, how would you set up the timer? Indicate your counting mode and show the value of ARR, CCR, and PSC registers.
4. Write an assembly program that uses the output compare function of a timer to toggle an LED every second.
5. Write an assembly program that generates a PWM output signal to dim an LED periodically.
6. Write an assembly program that uses PWM to generate a square wave signal with a frequency of 440 Hz and a duty cycle of 50% (*i.e.*, musical tone A).
7. Write an assembly program that uses PWM to control a stepper motor via micro stepping.
8. Write an assembly program that uses the input capture function to measure the frequency of an external signal. Use a function generator to generate a 1HZ square

wave. Send the square wave to the STM32L board to verify the correctness of your assembly program.

9. Write an assembly program that uses the HC-SR04 ultrasonic distance sensor presented in Chapter 15.4.3 to measure the distance to an object.
10. In the timer interrupt handler, we need to increment the overflow counter if an overflow occurs. Register `r3` holds the overflow counter. However, the main function cannot observe any changes in register `r3` even though overflow events have occurred. Explain why.

```
; r3 = overflow counter

TIM4_IRQHandler PROC
    EXPORT TIM4_IRQHandler
    ...

    LDR    r0, =TIM4_BASE
    LDR    r1, [r0, #TIM_SR]    ; Read status register
    AND    r2, r1, #TIM_SR_UIF ; Check update event flag
    CBZ    r2, check_CCIF      ; Compare and branch on zero

    ADDS   r3, r3, #1          ; Increment the overflow counter

check_CCIF
    ...
    ENDP
```